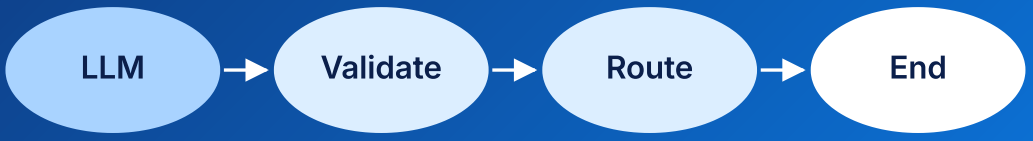
DMLS AGENTIC AI WORKSHOP · SESSION 1

From LLM to Agent

What companies actually hire for, built from scratch.

Student Guide

Every concept, every line of code, and help when you're stuck.
Follow along live, or work through it on your own later.



Dalhousie Machine
Learning Society

Wed Oct 14, 2026
Hosts: Akash Maity & Akshit Bhalla

How to Use This Guide

The slides give you the ideas in two hours. This guide gives you the rest: every concept explained properly, every notebook cell with its expected output, and a way back in if you fall behind.

Two ways to use it

During the session: keep it open next to Colab. Each chapter header shows the matching slide numbers and agenda time, so you can find your place in seconds. If a cell breaks, the expected output is printed right under the code.

After the session: read it front to back and type every cell yourself. Typing beats copy-paste; your fingers learn the shapes. Everything works on the free Groq tier.

Two tracks

The main text is the **base build**: everything you need to finish the project and explain it in an interview. The teal **Go deeper** boxes are the **upper-year track**: production upgrades and the "why" behind them. Skip them on a first read if you like. Their cells sometimes redefine notebook names, so try them in a copy (**File → Save a copy in Drive**) or re-run the original cell afterwards.

The boxes

CONCEPT An idea worth slowing down for.

PITFALL A mistake people actually make, and the fix.

GO DEEPER Upper-year: how production teams do it, with tested code.

CHECKPOINT Check yourself. Answers are in the separate Answer Key; the full lab solution is Appendix F.

Your manager asks / The interviewer asks cards mirror the deck's "At Work" slides: the same idea, framed the way it shows up on the job and in interviews. **Appendix D** shows the same skills in 17 real job postings; **Appendix E** puts every idea into one everyday picture.

New to Python? Start with **Chapter P**: only the Python this session uses, from zero, in five pages.

Slide → page map

SLIDES	AGENDA	TOPIC	PAGE
–	before we start	The Python you need (optional)	4
8	before we start	Setup: Groq key, Colab Secrets, test cell	9
9–22	0:00–0:15	Theory & scope: LLMs, the ceiling, agents, routing	11
23–24	0:15–0:25	Structured output	23
25–26	0:25–0:40	The state machine (Notebook Part 1)	26
27	0:40–0:50	Errors, retries and fallback	30
29–30	0:50–1:05	Branching, the reducer, observability (Part 2)	34
31	1:05–1:20	The LangGraph rebuild (Part 3)	40
32	1:20–1:30	Testing non-determinism	45
33	1:30–1:55	Your build: the lab	48
34	1:55–2:00	The Sentence	51

Contents

BEFORE WE START

P The Python You Need optional 4

0 Setup: Your Key, Colab, the Test Cell slide 8 9

PART 1 · THEORY & SCOPE

1 Why This Session slide 9 11

2 What an LLM Really Does slides 10, 13 12

3 The Ceiling: Where a Plain LLM Stops slides 11–12 16

4 Chatbot, Pipeline, Workflow or Agent? slides 14–15 17

5 What Is an Agent? slides 16–20, 22 19

6 What We're Building slide 21 21

PART 2 · THE BUILD

7 Structured Output slides 23–24 23

8 The State Machine slides 25–26 26

9 Errors, Retries and Fallback slide 27 30

10 Branching and the Reducer slide 29 34

11 Observability slide 30 37

12 The LangGraph Rebuild slide 31 40

PART 3 · PROVE IT

13 Testing Non-Determinism slide 32 45

14 Lab: Add the Urgent Path slide 33 48

PART 4 · WRAP-UP & REFERENCE

15 The Sentence slide 34 51

16 Question Bank: Manager & Interview all At Work slides 52

APPENDICES

A Glossary 54

B Common Errors Cheat-Sheet 55

C Further Reading 56

D What Employers Actually Ask For 17 real postings 57

E Explain It Simply analogy cheat-sheet 60

F Lab Solution: The Urgent Path read after you try 61

CHAPTER P | BEFORE SLIDE 8 | BEFORE WE START

The Python You Need

You don't need to be a Python expert today. You need about fifteen ideas, and every one of them shows up in the notebook. This chapter explains each one from zero, with a tiny example and a pointer to where you'll meet it.

CHECK YOURSELF

Read these four lines. If you can say what each one does, you know enough Python for today and can skip to Chapter 0. If not, the sections below explain every piece.

1. `handlers = {"billing": handle_billing}`
2. `state.trace.append(f"route: {state.category}")`
3. `return handlers[state.category](state)`
4. `category: Optional[Literal["billing", "technical"]] = None`

Values	Containers	Code that runs	Shapes for data
`str` "T-001"	`list` the trace	`if` / `elif` / `else`	`class` + BaseModel
`int` / `float` 0.92	`dict` the routing table	`for` loops	type hints
`bool` True	`f"..."` trace lines	`def` functions	`Literal` / `Optional`
`None` not decided yet		`try` / `except`	`TypedDict`

Figure P.1 *Every Python idea this session uses. Left to right: values, containers, logic, then the data shapes Pydantic and LangGraph build on.*

P.1 · What happens when you run a cell

A notebook is a page of **cells**. When you press **Shift + Enter**, Colab sends that cell to a Python program running on a Google computer (the **kernel**), which runs it top to bottom and shows anything it prints. Two things trip up almost everyone:

- **Order matters, not position.** The kernel remembers everything you ran, in the order you ran it. If you redefine `TicketState` in Chapter 10, the new one wins, even if you scroll back up. When things get confusing, use **Runtime → Restart and run all**.
- **A crashed cell stops where it crashed.** The lines after the error never ran, so the names they define don't exist yet. That's what `NameError` usually means.

THINK OF IT LIKE THIS

The kernel is a chef with a notepad. Each cell you run is an instruction slip. The chef remembers every slip in the order it arrived, not the order the slips are pinned on the wall.

P.2 · Values and variables

A **variable** is a name that points at a value. You create one with `=`.

```
ticket_id = "T-001"      # str: text, always in quotes
confidence = 0.92        # float: a number with a decimal point
attempts = 3             # int: a whole number
validated = True         # bool: True or False, capital first letter
response = None          # None: "no value yet"
```

`None` deserves attention. Many state fields start as `None` because the agent hasn't decided them yet: `category = None` means "intake hasn't run", not "the category is empty".

P.3 · Strings and f-strings

The notebook does three things with text: lowercase it, check what's inside it, and build new text from values.

```
text = "My server keeps CRASHING"
text.lower()                # "my server keeps crashing"
"crash" in text.lower()     # True: is this text inside that text?
len("  hi  ".strip())        # 2: strip() removes spaces at both ends
```

An **f-string** (a string with an `f` in front) drops values into text with `{curly braces}`. Every trace line in the agent is built this way. `:.2f` means "two decimal places".

```
category, confidence = "billing", 0.9234
f"intake: classified as '{category}' (confidence={confidence:.2f})"
# "intake: classified as 'billing' (confidence=0.92)"
```

P.4 · Lists and dictionaries

A **list** is an ordered collection. The agent's trace is a list of strings; `.append()` adds one to the end. Counting starts at 0.

```
trace = []
trace.append("validate: passed")
trace.append("respond: handled by billing")
trace[0]          # "validate: passed"
```

A **dictionary** maps keys to values. The notebook uses one as a **dispatch table**: a dict from category to the function that handles it. This is the heart of routing in Chapter 8.

```
handlers = {"billing": handle_billing, "technical": handle_technical}
handlers["billing"]          # the handle_billing FUNCTION itself, not called yet
handlers["billing"](state)   # ...and now we call it
```

KEYERROR

`KeyError: 'urgent'` means you asked a dict for a key it doesn't have. Remember this one: it's the bug most people hit in the lab (Chapter 14).

In LangGraph (Chapter 12) the state *is* a dict, and you'll see `{**state, "response": reply}`. It means "copy everything from `state`, then set this one key". It builds a new dict and leaves the old one alone.

P.5 · Decisions and loops

```
t = text.lower()
if "invoice" in t or "charge" in t:
    cat = "billing"
elif "error" in t or "crash" in t:
    cat = "technical"
else:
    cat = "general"
```

Python checks each condition from the top and runs the **first** branch that's true. Order matters: in the lab you'll check for urgent words first, because an outage ticket often also contains "crash". `any(w in t for w in ["lawsuit", "lawyer"])` means "is at least one of these words in the text?" (Chapter 10).

Loops

```
for attempt in range(1, 4):     # attempt = 1, then 2, then 3
    print("trying, attempt", attempt)
```

`range(1, 4)` stops *before* the second number. The retry loop in Chapter 9 is exactly this shape. If your trace ever shows `{category}` instead of `billing`, you forgot the `f` on an f-string.

P.6 · Functions

A **function** is a named, reusable block of code. It takes inputs and hands back an output with `return`. Every node in the agent has the same shape: **state in, state out**.

```
def validate(state):
    if len(state.text.strip()) < 5:
        state.validated = False
    else:
        state.validated = True
    return state
```

- **Default values:** `def classify_with_retry(text, max_attempts=3)` means you can call it with just the text, and `max_attempts` will be 3.
- **Functions are values.** You can store one in a variable or a dict, pass it to another function, and call it later. Dispatch tables (P.4) and decorators (P.10) both depend on this.

FORGOT RETURN

A node without `return state` hands the next node `None`, which fails with `AttributeError: 'NoneType' object has no attribute …`.

P.7 · Classes, objects and Pydantic

A **class** is a blueprint; an **object** is one thing built from it. Every class you write today is a **Pydantic model**: a class that describes the *shape* of some data and refuses data that doesn't fit.

```python
from typing import Literal
from pydantic import BaseModel

class TicketClassification(BaseModel):
    category: Literal["billing", "technical", "general"]
    confidence: float
    reasoning: str

c = TicketClassification(category="billing", confidence=0.9, reasoning="mentions an invoice")
c.category      # "billing": read fields with a dot
TicketClassification(category="refunds", confidence=0.9, reasoning="x")
# ValidationError: Input should be 'billing', 'technical' or 'general'
```

Read each field like a sentence: **category must be exactly one of these three strings**, **confidence is a number**, **reasoning is text**. That `ValidationError` isn't a nuisance, it's the point. Chapter 7 is built on Pydantic refusing answers that don't fit.

THINK OF IT LIKE THIS

A Pydantic class is a web form with dropdowns instead of blank boxes. Nobody can type "kinda billing?" into a dropdown.

P.8 · Type hints, Optional, Literal, TypedDict

The `: str` and `→ TicketState` bits are **type hints**: they say what kind of value something should be. `def intake(state: TicketState) → TicketState` reads "intake takes a TicketState and returns one". In plain Python, hints are notes for humans and your editor. In Pydantic classes they become **rules that are enforced**.

HINT	MEANS	WHERE YOU'LL SEE IT
`Optional[float]`	a float, or `None`	State fields that start empty (Ch. 8)
`Literal["normal", "high"]`	exactly one of these values	Categories and priority (Ch. 7, 10)
`list[str]`	a list of strings	The trace (Ch. 8)
`TypedDict`	a dict with declared keys and value types	LangGraph state (Ch. 12)

P.9 · Errors: try and except

When something goes wrong, Python **raises an exception** and stops, unless you catch it:

```python
try:
    result = classify_with_llm(text)        # might fail: network down, rate limit
except Exception as e:
    print("that failed:", e)                # runs only if the try block failed
```

Chapter 9 builds its whole retry system on this. Catching every `Exception` is fine for learning, but it also catches *your own* bugs (a typo becomes a silent fallback). Chapter 9's Go deeper box shows how production code retries only the errors that are worth retrying.

P.10 · Decorators, in one paragraph

A **decorator** is a function that takes a function and returns an upgraded version of it. You'll meet one in Chapter 11. It can be written two ways, and they mean the same thing:

```
intake = traced_node("intake")(intake)      # the way the notebook does it

@traced_node("intake")                       # the same thing, written with @
def intake(state): ...
```

P.11 · Imports, packages and JSON

`import json` loads a module that ships with Python. `!pip install ...` (Chapter 0) downloads the ones that don't:

PACKAGE	WHAT IT DOES FOR US
`groq`	Talks to Groq's API: sends your messages, brings back the model's reply
`pydantic`	Defines data shapes and validates data against them
`instructor`	Makes the model's answer fit a Pydantic class, and re-asks when it doesn't
`langgraph`	Builds the agent as a graph of nodes and edges
`ipytest`	Runs pytest tests inside a notebook

JSON is the text format APIs use for structured data. It looks almost exactly like a Python dict, and `json.dumps(d)` turns a dict into a JSON string, which is how Chapter 11 writes its log lines.

P.12 · What an API call actually is

An **API** is a door into someone else's program. When the notebook "calls the model", your code sends a message over the internet to Groq's servers; a model running there reads it and sends back a reply. Your **API key** is the password that proves the request is yours, so Groq can apply your free limits.

THINK OF IT LIKE THIS

An API call is a drive-through. You speak into the box (the request), a kitchen you can't see does the work, and the order comes out the window (the response). The API key is your loyalty card: it's how they know whose account it is, even when the meal is free.

GO DEEPER · TOKEN BUDGETS

Free tiers limit **tokens per minute** as well as requests, and some count the *maximum* a request could write, not what it actually writes. If a one-line question fails with `429 … Request too large … output tokens per minute`, pass `max_tokens=250` to the call. A ticket label needs fewer than 100 tokens, so nothing is lost, and every request fits inside the per-minute budget. Job posts call this skill "token budgets" (Appendix D).

CHAPTER 0 SLIDE 8 BEFORE WE START

Setup

Free, no credit card, about five minutes. Do this once, before the session if you can.

Step 1 · Get a free Groq API key

1. Go to **console.groq.com** and sign in (Google, GitHub or email).
2. Open **API Keys** → **Create API Key**. Give it a name like `dmls-workshop`.
3. Copy the key now. It starts with `gsk_` and is shown **only once**. If you lose it, delete it and make a new one.

Step 2 · Store it in Colab Secrets, not in your code

1. Open the workshop notebook in Google Colab (use the link the hosts share). No link? **File** → **New notebook** works too: type the cells from this guide.
2. Click the **key icon** in the left sidebar (the **Secrets** panel) → **Add new secret**.
3. Name: `GROQ_API_KEY` (exactly, capitals and all). Value: paste your key.
4. Switch on **Notebook access**. The first time a cell reads the secret, Colab may ask you to grant access; allow it.

NEVER PASTE THE KEY INTO A CELL

Notebooks get shared and pushed to GitHub, and bots scan public repos for keys within minutes. A secret stays with *your* Colab account, so anyone you share the notebook with gets the code, not your key. If a key ever leaks, delete it in the Groq console.

Step 3 · Install and connect

Run the install cell first. The `!` means "run this as a shell command, not Python". The `~=` pins each library to the minor version this guide was tested with (for example, any 1.17.x of Instructor).

NOTEBOOK CELL · INSTALL — main/m00_install.sh

```
!pip install -q "instructor~=1.17.0" "groq~=1.7.0" "langgraph~=1.2.0" "ipytest~=0.14.0"
```

Then the setup cell. Every later cell uses the three names it creates: `MODEL`, `groq_client` and `client`.

NOTEBOOK CELL · SETUP — main/m01_setup.py

```
import os
from google.colab import userdata                 # Colab's Secrets panel (the key icon)
os.environ["GROQ_API_KEY"] = userdata.get("GROQ_API_KEY")

import instructor
from groq import Groq

MODEL = "llama-3.3-70b-versatile"                 # retired? see console.groq.com/docs/models

groq_client = Groq()                              # reads GROQ_API_KEY from the environment
client = instructor.from_groq(groq_client)        # same client, now it also accepts response_model=
```

- **Lines 1–3** read your secret and put it in an environment variable, which is where the Groq library looks for it.
- `MODEL` is one constant so you can switch models in one place. `llama-3.3-70b-versatile` is a production model on Groq and supports tool calling, which Chapter 7 relies on.
- `groq_client` is the plain Groq client: text in, text out.

- `client` is the same client wrapped by **Instructor**, which adds the `response_model=` argument. That one argument is the subject of Chapter 7.

Step 4 · Run the test cell

NOTEBOOK CELL · TEST main/m02_ready.py

```
reply = groq_client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "user", "content": "Reply with exactly one word: ready"}],
)
print(reply.choices[0].message.content)
```

OUTPUT example: the model's wording and numbers will differ

```
ready
```

An answer within a few seconds means you're ready. Anything else, check the table below.

One key each. Free-tier limits (requests and tokens per minute, plus a daily cap) are per account. Thirty people on one key will hit them in five minutes.

Setup troubleshooting

YOU SEE	WHAT IT MEANS AND THE FIX
`SecretNotFoundError`	The secret name doesn't match. It must be exactly `GROQ_API_KEY`.
`NotebookAccessError`	Notebook access is switched off for that secret. Toggle it on and re-run the cell.
`ModuleNotFoundError: instructor`	The install cell hasn't run in this session. Run it, and if Colab asks you to restart the session, restart and run from the top.
`Error code: 401 … invalid_api_key`	The key is wrong, revoked, or has a stray space. Copy it again or create a new one.
`Error code: 429 … rate_limit_exceeded`	You hit a free-tier limit. Wait a minute. Make sure you aren't sharing a key or running a cell in a loop.
`model_decommissioned` or `404`	Groq retired the model. Pick a current one from **console.groq.com/docs/models** (for example `openai/gpt-oss-20b`) and change `MODEL`.
`NameError: name 'client' is not defined`	The setup cell didn't run in this session (Colab forgets everything on restart). Run it again.

GO DEEPER · OTHER PROVIDERS AND RUNNING LOCALLY

After the setup cell, everything goes through `client` and `MODEL`, so the provider is a two-line change. Only the Groq path was tested end to end for this guide; check Instructor's docs for the current setup of the others.

- **OpenAI:** `client = instructor.from_openai(OpenAI())`, `MODEL = "gpt-4o-mini"` (paid key).
- **Local model with Ollama:** `instructor.from_openai(OpenAI(base_url="http://localhost:11434/v1", api_key="ollama"), mode=instructor.Mode.JSON)`, with `MODEL` set to a model you've pulled. It's slower, but it's free and works offline.
- **Not using Colab?** Run `pip install` without the `!`, set the key in your terminal (`export GROQ_API_KEY=gsk_…` on macOS/Linux), and delete the two `userdata` lines.

PART ONE

Theory & Scope

What a language model actually does, why it can't be trusted on its own, and the shape of the system that fixes that: an agent that is mostly plain, testable code with the model plugged in only where judgement is needed.

1 · Why this session 2 · What an LLM really does 3 · The ceiling 4 · Chatbot, pipeline, workflow or agent?
5 · What is an agent? 6 · What we're building

CHAPTER 1 SLIDE 9 0:00–0:15

Why This Session

Most of today is software engineering around an unreliable component, not prompting. That's the part companies hire for.

Read a few job posts for "AI engineer", "LLM engineer" or "applied ML engineer" and the same four skills keep showing up. None of them is "writes clever prompts". They're the skills you need to put a model that is *usually* right inside a product that has to be right *every time*.

1 · Agent frameworks

LangGraph and friends, and knowing what they actually do for you. You'll build the agent by hand first, then port it, so the framework is code you already understand, not magic.

2 · Structured output

Forcing an unreliable model to return data your code can trust: a typed object with a fixed set of allowed values, validated on every call.

3 · Observability & evals

Answering "why did it do that?" with evidence: a trace of what every step saw, decided and how long it took, plus tests that hold up when the model's answers wobble.

4 · Failure handling

Planning for the call that fails, because it will. Rate limits, outages, bad keys, malformed answers: the system degrades gracefully instead of crashing.

What you'll be able to do by the end

- Explain what an LLM is doing in plain language, and why that makes it hallucinate.
- Tell a chatbot, a pipeline, a workflow automation and an agent apart, and pick the simplest one that works.
- Build a stateful **customer-support triage agent** from scratch: structured output, a state machine, retries with a fallback, branching checks and a trace.
- Rebuild the same agent in **LangGraph** and say exactly what the framework added.
- Test code whose output isn't the same twice, and defend which parts you tested and how.
- Say one specific, accurate sentence about what you built (Chapter 15) in a resume or an interview.

THE MINDSET FOR TODAY

Treat the LLM like a brilliant but unreliable colleague. It's great at judgement calls on messy text, and it will sometimes be confidently wrong, slow, or unavailable. Your job is to design the system so that none of those failures reach the customer. Every chapter in Part 2 is one layer of that design.

What you need to know already

Basic Python: functions, classes, dictionaries, lists and `try/except`. You don't need prior ML or agent experience. Where the guide uses something slightly advanced (type hints, `Literal`, decorators, `TypedDict`), it explains it in place.

CHAPTER 2 SLIDES 10 & 13 0:00–0:15

What an LLM Really Does

A large language model does one thing: given some text, it predicts the next piece of text. Five ideas explain almost everything else, including why it hallucinates.

1 · Tokens

Models don't read letters or words. They read **tokens**: chunks of text, roughly a word or part of a word. Before your prompt reaches the model, a *tokenizer* turns it into a list of numbers, one per token. Common words are usually one token; rare or long words get split. In English, a token averages about three-quarters of a word.

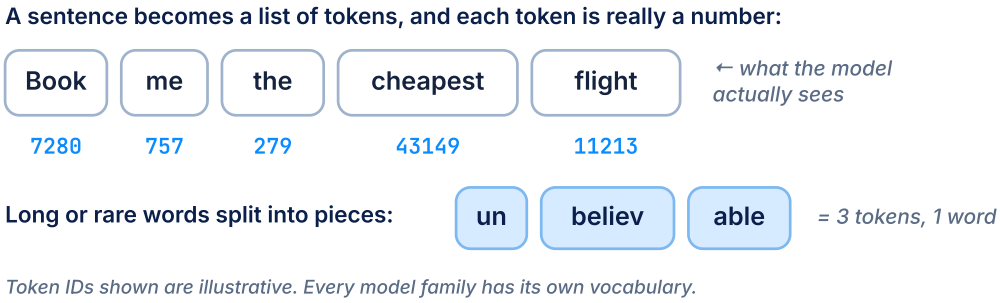


Figure 2.1 *What the model actually receives: token IDs, not words.*

Tokens matter because **everything is counted in them**: how much text fits in one call, how fast the answer comes back, what a paid API charges, and every rate limit on your free key.

2 · One token at a time

The model reads all the tokens so far and produces a score for *every* token in its vocabulary (Llama 3's has about 128,000). Those scores become probabilities. One token is picked, appended to the text, and the whole thing repeats until the model emits a "stop" token. A long answer is hundreds of these tiny predictions in a row.

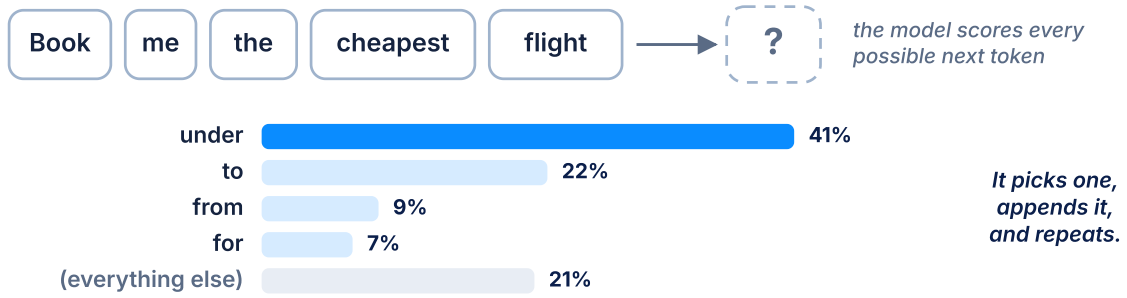


Figure 2.2 *Each step is a probability distribution over the next token. "under" is likely here because of the patterns the model learned in training, not because it checked a flight.*

3 · Temperature

Temperature controls how random that pick is. Mathematically, each probability is raised to the power 1/T and re-normalised. A low temperature sharpens the distribution so the top token almost always wins. A high one flattens it, so unlikely tokens get picked more often.



Figure 2.3 *The same five candidates at three temperatures. These are real numbers: each bar is $p^{1/T}$, re-normalised.*

For **classification**, which is what we do today, you want low temperature: the same ticket should get the same label. For brainstorming or creative writing you might want more variety. Even at temperature 0, outputs can differ slightly between runs (hardware and batching effects), so **never write code that depends on the exact wording**. Chapter 13 shows how to test around that.

4 · The context window

The **context window** is everything the model can see during one call: your instructions, the conversation so far, any documents you paste in, and the new message. It's measured in tokens. `llama-3.3-70b-versatile` on Groq has a 131,072-token window, roughly a few hundred pages. If something isn't in the window, then for the model **it doesn't exist**: not your database, not today's prices, not your company's refund policy.

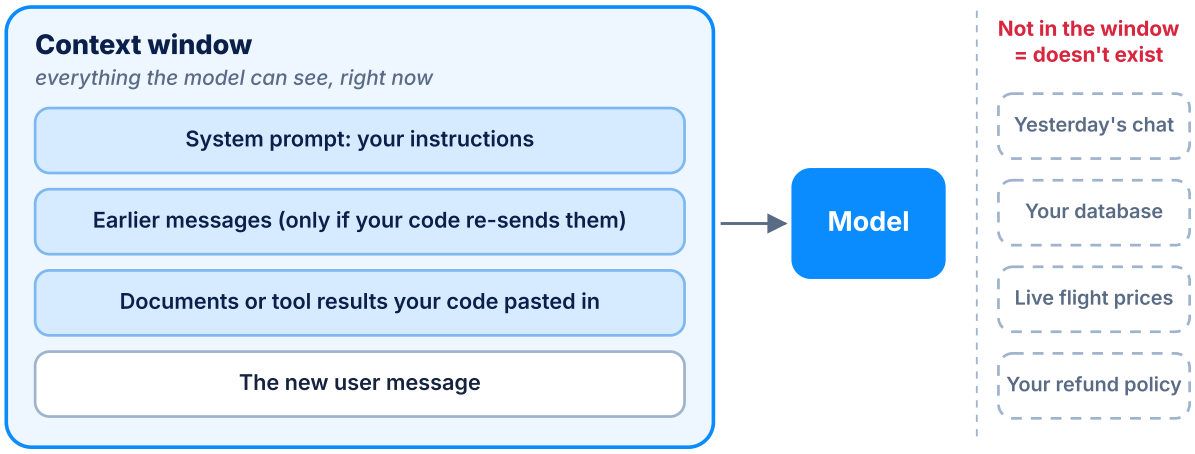


Figure 2.4 *Your code decides what goes into the window. That decision is most of the engineering.*

5 · No memory between calls

Each API call starts from zero. A chatbot that seems to remember you is really your code re-sending the whole conversation every turn. Try it:

TRY IT · STATELESSNESS main/m04_memory.py

```python
def ask(messages):
    reply = groq_client.chat.completions.create(model=MODEL, messages=messages)
    return reply.choices[0].message.content

# Two separate calls: the second one has never heard of Priya
print(ask([{"role": "user", "content": "Hi! My name is Priya."}]))
print(ask([{"role": "user", "content": "What's my name?"}]))

# The fix: YOUR code carries the conversation forward
history = [
    {"role": "user", "content": "Hi! My name is Priya."},
    {"role": "assistant", "content": "Nice to meet you, Priya!"},
    {"role": "user", "content": "What's my name?"},
]
print(ask(history))
```

OUTPUT example: the model's wording and numbers will differ

```
Nice to meet you, Priya! How can I help you today?
I don't have any way of knowing your name. This is the start of our conversation, so you haven't told me yet.
Your name is Priya.
```

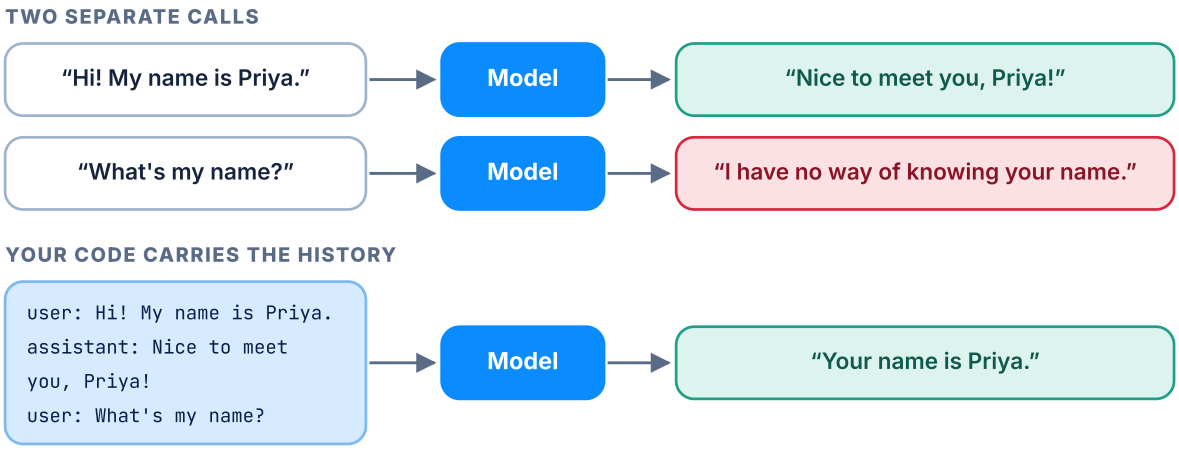


Figure 2.5 *"Memory" is a list your code keeps and re-sends. In our agent, that list is replaced by a typed* **state object** *(Chapter 8).*

So why do LLMs hallucinate?

Put the five ideas together. The model produces the **most plausible next token** given what's in its window. Plausible isn't the same as true. When the right fact isn't in the window, or wasn't in its training data, the model doesn't stop. It keeps producing fluent, confident, plausible text, because that's the only thing it does. There's no internal "I checked a source" step unless your system adds one.

Your manager asks

"Can we just let the bot answer the refund questions?"

- It predicts text. It doesn't know our refund policy.
- Ground it: put the real policy in its context.
- Anything about money that isn't in the policy goes to a human.

The interviewer asks

"Why do LLMs hallucinate?"

- They predict likely tokens, not facts.
- With no source to check, plausible wins.
- The fix: ground it, constrain it, verify it.

CHECKPOINT

1. Why does a long answer take longer to generate than a short one?
2. You ask the model about your company's internal wiki and it answers confidently. Should you trust it? Why or why not?
3. Which temperature would you choose for routing support tickets, and why?

CHAPTER 3 SLIDES 11–12 LIVE DEMO

The Ceiling

Ask a plain LLM to do something in the real world and watch where it hits its limit.

LIVE DEMO · BOOK ME A FLIGHT main/m03_ceiling.py

```
reply = groq_client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "user", "content": "Book me the cheapest flight under $400."}],
)
print(reply.choices[0].message.content)
```

OUTPUT example: the model's wording and numbers will differ

```
I'd be happy to help you find a cheap flight! However, I'm a text-based assistant and I can't search live flight
prices or make bookings.

To find the cheapest flight under $400, try Google Flights, Kayak or Skyscanner and set a price alert. Could you tell
me your departure city, destination and travel dates? I can suggest tips for finding a good fare.
```

You'll see one of two things. Either a polite refusal ("I can't book flights...") or, worse, a confident invention: a flight number, a price, a booking reference, none of them real. Both are failures for a product. A refusal does nothing, and an invention does something wrong.

Why it failed

No memory

It can't recall anything beyond what's in front of it: your past trips, your home airport, what you said yesterday.

No goals

It answers once, then stops. There's no plan and no follow-through: search, compare, confirm, book.

No ability to act

It can't search, check prices, or book anything. It can only produce text *about* those things.

Each missing piece maps onto a part of an agent (Chapter 5): memory becomes **state**, goals become a **controller** that loops until done, and the ability to act becomes **tools** plus code that executes decisions.

A real case: the chatbot that invented a refund policy

In **2022**, a passenger asked Air Canada's website chatbot about bereavement fares after a death in the family. The chatbot said they could buy a full-price ticket now and claim the bereavement discount afterwards, within 90 days. The airline's actual policy page said the opposite: no retroactive claims. When the passenger applied, Air Canada refused.

In **February 2024**, British Columbia's Civil Resolution Tribunal ruled in the passenger's favour (*Moffatt v. Air Canada*). Air Canada had argued it wasn't responsible for what its chatbot said; the tribunal rejected that and held the airline responsible for all the information on its website, chatbot included. It was ordered to pay the fare difference plus interest and fees.

THE LESSON

The chatbot did exactly what an LLM does: it produced a plausible answer. The failure was the **system design**. The real policy wasn't in its context, nothing constrained what it could promise, and nobody verified its answer before a customer acted on it. You can't make the model never wrong, but you can design the system so its wrong answers can't hurt anyone.

The fix, in three words

FIX	WHAT IT MEANS	WHERE YOU'LL DO IT
Ground it	Put the real facts (policy, data, documents) into the context window instead of hoping the model knows them.	Retrieval (RAG), later in the series
Constrain it	Limit what the model is allowed to output, and let plain code decide what happens next.	Structured output (Ch. 7) and the state machine (Ch. 8)
Verify it	Check the output before anything acts on it: validation, rules, tests, and a human for high-stakes calls.	Validation (Ch. 7–8), tests (Ch. 13), human-in-the-loop in Session 2

Slide 13 sums this up: "Every chapter today is a fix for something on this slide."

CHAPTER 4 SLIDES 14–15 SCOPE CHECK

Chatbot, Pipeline, Workflow or Agent?

"Agent" is the most overused word in AI right now. Before building one, be clear about what it is, what it isn't, and when you shouldn't build one at all.

	WHAT IT DOES	USE IT WHEN	EXAMPLE	TODAY?
Chatbot TEXT OUT	Answers in text. A person reads it and decides what to do.	Someone just needs an answer or a draft.	Help-centre chat, FAQ bot	No
Pipeline FIXED PATH	The same steps in the same order, every time: A → B → C.	Inputs are predictable and the steps never change.	Nightly job: pull data → clean → email a report	No
Workflow automation IF / THEN	When an event fires, run fixed if/then rules. No judgement involved.	The decision fits a simple rule.	Form submitted → add a row → Slack ping (think Zapier)	No
Agent DECISION OUT	The LLM judges the case; your code acts on that judgement.	The input is messy text and the next step depends on it.	Today's ticket triage	**Yes**

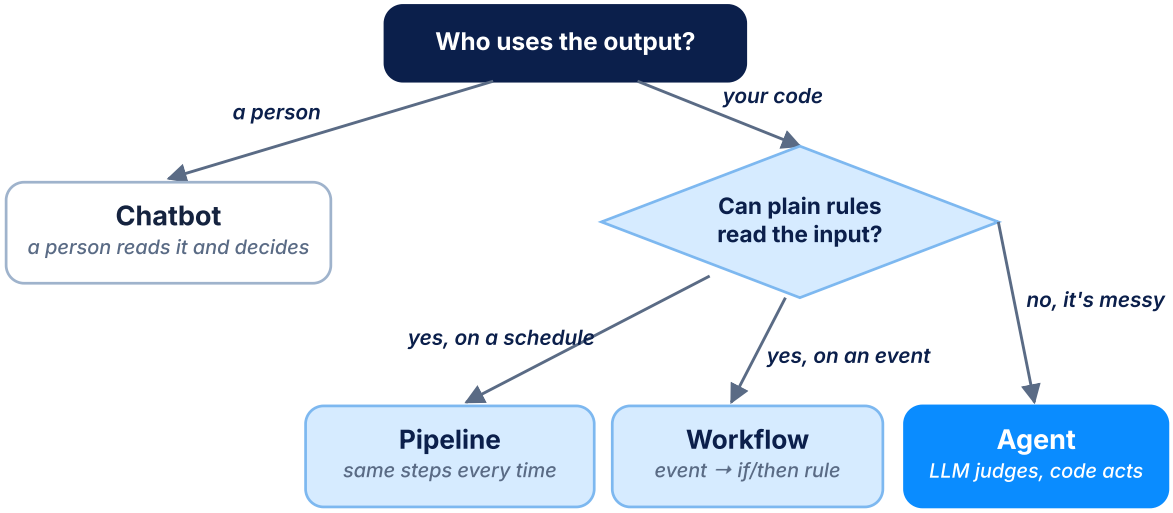


Figure 4.1 *Which one, when? Each step to the right adds flexibility and adds cost, latency and risk.*

The rule of thumb

Pick the simplest one that works. Move right only when the rules can't read the input. If a regular expression or a lookup table can make the decision, use it: it's free, instant and never hallucinates. Bring in an LLM only for the part that genuinely needs judgement over messy, human text. Here that's "is this ticket about billing, a technical fault, or something else?"

Anthropic's engineering guide to building agents makes the same point: start by *"finding the simplest solution possible, and only increasing complexity when needed."* You'll hear this idea again on slide 22, and again in every interview about agents.

WHAT WE'RE BUILDING TODAY, PRECISELY

An agent in the **reliable** sense: routing plus state. The LLM makes one judgement (the category), and deterministic code does everything else. We deliberately skip the flashy "let the model loop until it decides it's done" style. It demos well and breaks in production, and it isn't what most companies ship. Chapter 5 explains that split.

EXERCISE · CLASSIFY SIX REAL REQUESTS

For each one, pick chatbot, pipeline, workflow automation or agent, and say why in one sentence. Answers are in the Answer Key.

1. A widget on the help page that answers "What are your opening hours?"
2. Every night at 2 a.m., export yesterday's orders to a spreadsheet and email it to finance.
3. When a Google Form is submitted, add a row to a sheet and post a message in Slack.
4. Read free-text expense notes and decide which ones need a manager's approval.
5. Every image uploaded to a bucket is resized into three standard sizes.
6. Read incoming emails and send each to sales, support or spam.

CHAPTER 5 SLIDES 16–20, 22 0:00–0:15

What Is an Agent?

An agent is an LLM given agency. Instead of producing an answer for *you*, it produces a decision for *the system*, and code acts on it.

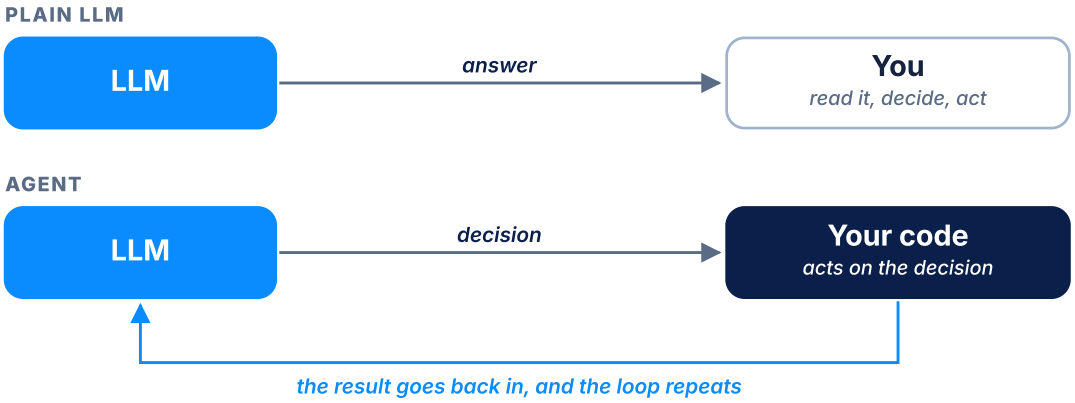


Figure 5.1 *The whole difference: who consumes the output. A person reads an answer; code executes a decision.*

Once code is consuming the model's output, two things change. First, the output has to be **machine-readable**: your code can't act on "Uhh... probably technical?" (Chapter 7 fixes that). Second, there's a **loop**: the code acts, the result changes the situation, and the system decides again.

The agent loop

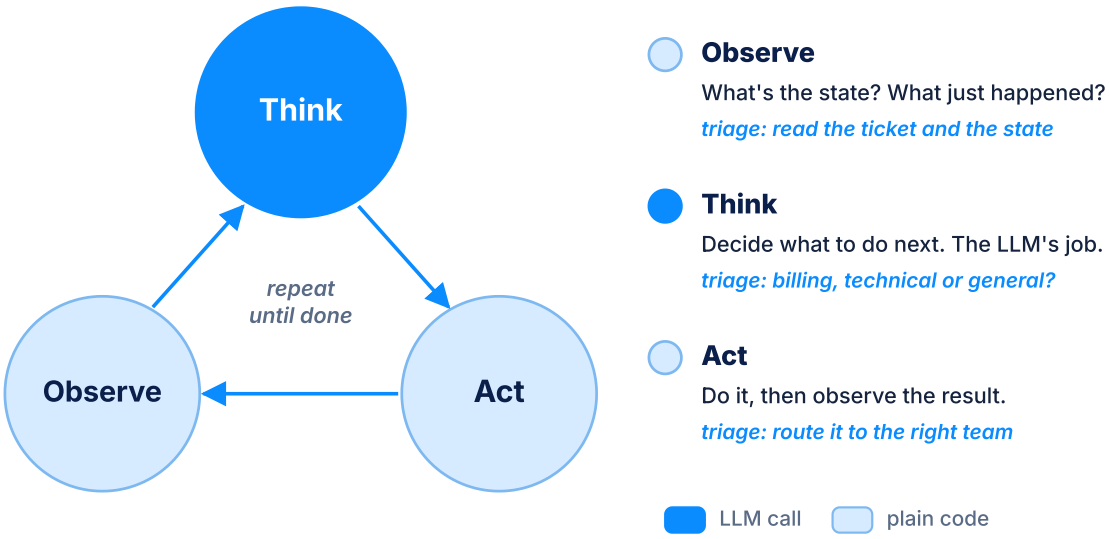


Figure 5.2 *Observe, think, act, repeat until done. Only "think" is an LLM call; the rest is ordinary code. This diagram comes back all series.*

Our triage agent goes around this loop exactly once per ticket: read the ticket (observe), classify it (think), route it (act). Agents in later sessions go around many times: think, call a tool, observe the result, think again.

Core components

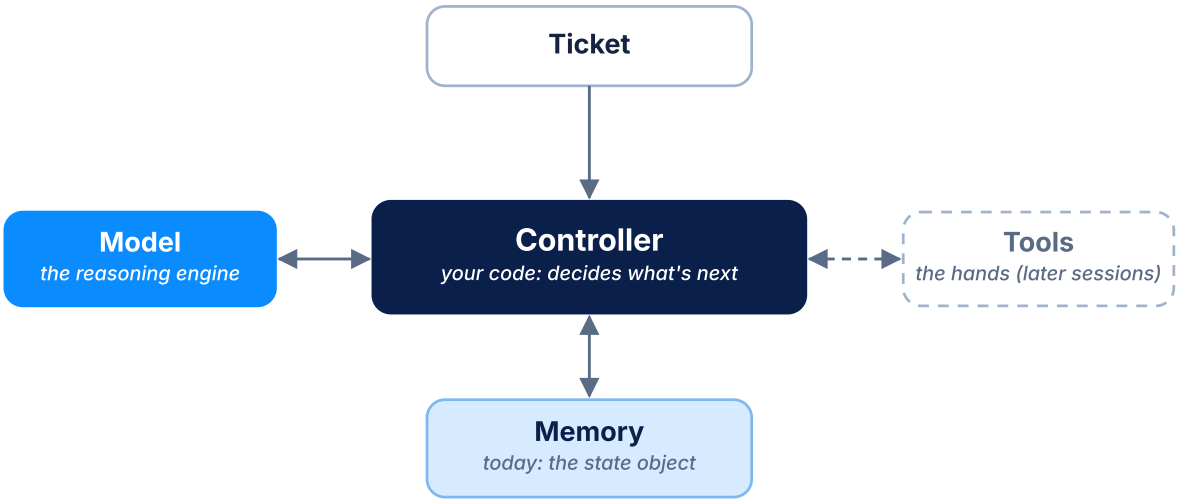


Figure 5.3 *The four parts of any agent. Today you build the controller and the memory; the model is rented from Groq.*

- **Model:** the reasoning engine. It makes the judgement calls and nothing else.
- **Tools:** the hands, meaning functions the agent can call (search, look up an order, send an email). Not built today; they arrive later in the series.
- **Memory:** what the agent knows about the current job. Today that's the **state object**: one typed record that every step reads and writes.
- **Controller:** the code that decides what happens next. **You build this.** It's where reliability lives.

The split nobody talks about

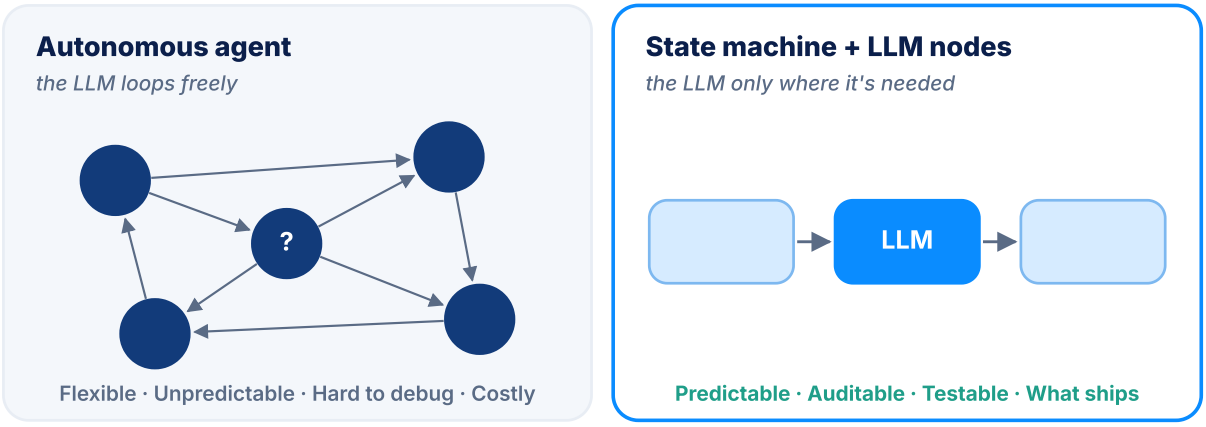


Figure 5.4 *Two very different things are both called "agents".*

An **autonomous agent** hands the model the steering wheel: it decides which step comes next, over and over, until it decides it's finished. That's flexible, and it's also unpredictable, expensive, and very hard to debug, because the path is different every run.

Most systems companies actually ship are the right-hand picture: a **state machine with an LLM in specific nodes**. Engineers write the path; the model fills in the one decision rules can't make. It's predictable, auditable and testable. Anthropic's guide draws the same line, calling the first kind **agents** (the LLM directs its own process) and the second kind **workflows** (LLMs orchestrated through predefined code paths), and recommends starting with workflows. By that definition, today's agent is a **routing workflow**: code fixes the path and the LLM makes one decision. Don't confuse it with *workflow automation* from Chapter 4, which has no LLM at all.

Routing: the router and the rails

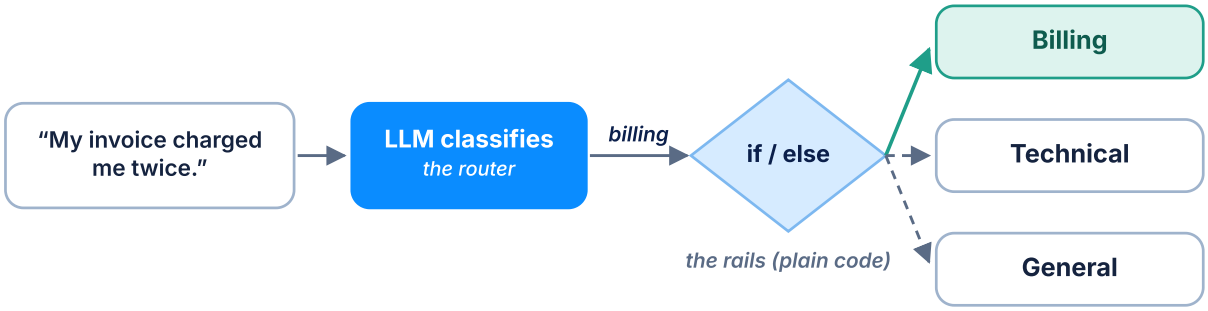


Figure 5.5 *The LLM only classifies. Ordinary* `if/else` *or a dictionary does the routing.*

We never let the model execute actions directly. We ask it a narrow question ("billing, technical or general?"), get a structured answer back, and let deterministic code move the ticket. The LLM is the **router**; your code is the **rails**. If the model is wrong, the worst case is a ticket in the wrong queue, not a refund the company never offered.

Your manager asks

"Let's build a fully autonomous support agent."

- Map the steps. Are they known in advance?
- If yes: a state machine, with the LLM in one node.
- Autonomy only where you can't write the path.

The interviewer asks

"When would you *not* build an agent?"

- When the steps are known up front.
- Workflows are cheaper, faster and testable.
- Agents trade all of that for flexibility.

GO DEEPER · THE CLASSIC AGENT TYPES

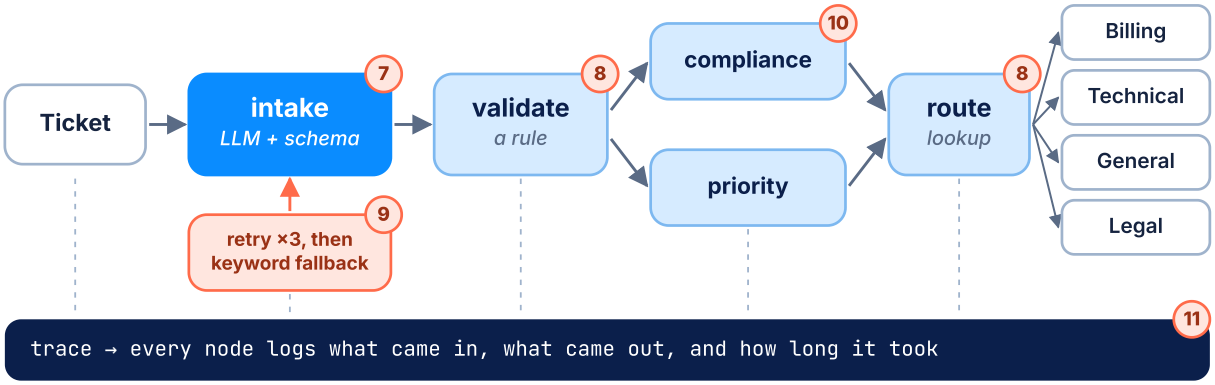
Textbooks (Russell & Norvig) sort agents by how they choose actions: **reactive** (condition → action), **model-based** (keeps internal state), **goal-based** (plans towards a goal) and **utility-based** (weighs trade-offs to pick the best action). Today's agent is reactive with state, and its "condition" is judged by an LLM. Planning and search come later in the series.

CHAPTER 6 SLIDE 21 THE BLUEPRINT

What We're Building

A customer-support triage agent, built from scratch in plain Python, then ported to LangGraph. Here's

the finished shape before we type a line of it.



Chapter 12 rebuilds this exact graph in LangGraph. Chapter 13 tests it. Orange tags = chapter numbers.

Figure 6.1 *The end state of today's build. Bright blue is the only LLM call; everything else is plain code.*

A ticket comes in as free text. **Intake** asks the LLM for a category, confidence and reasoning, forced into a schema, retried if the call fails, and backed by a keyword fallback if every try fails. **Validate** rejects tickets too short to act on. Two independent checks flag **compliance** risks (legal words) and **priority** (urgent words). **Route** is a lookup table that sends the ticket to the right team. Every node writes to a **trace**, so any decision can be explained afterwards.

HOW TO FOLLOW ALONG IN THE NOTEBOOK

- **Run cells top to bottom.** Later cells redefine earlier names on purpose (`intake` gets upgraded twice, `TicketState` once). Running out of order gives you a mix of old and new versions.
- **Lost?** Choose **Runtime → Run before** on the cell you're on to rebuild everything above it. After a restart, Colab forgets every variable, so re-run from the setup cell.
- **Read the output, not just the code.** Each listing in this guide shows what it printed when it was tested. If yours looks structurally different (missing lines, an error), stop and compare before moving on.
- **LLM outputs will differ** from the printed examples in wording and confidence numbers. That's expected, and it's exactly why Chapter 13 exists.

PART TWO

The Build

Six layers, one at a time, each fixing a specific way LLM systems fail in production. By the end of this part you'll have the same agent twice: once wired by hand, and once in LangGraph.

7 · Structured output
8 · The state machine
9 · Errors & retries
10 · Branching & the reducer
11 · Observability
12 · The LangGraph rebuild

CHAPTER 7 · SLIDES 23–24 · 0:15–0:25 · LIVE CODE

Structured Output

Never parse free text with string matching. Force the model into a schema, so your code receives an object it can trust.

✗ Hoping

"Uhh... probably technical? Could be billing tbh."

Free text. Your code has to guess, with `if "billing" in reply` string matching. What if it says "Billing", "BILLING:", "not billing", or wraps the answer in a paragraph?

✓ Knowing

category → one of billing · technical · general
confidence → a number from 0.0 to 1.0
reasoning → why it decided

A schema, validated on every call, and retried automatically if the response doesn't match.

Pydantic in 60 seconds

Pydantic turns a Python class with type hints into a validator. You declare the shape once; Pydantic checks every value against it and either gives you a clean object or raises a `ValidationError` explaining exactly what's wrong. `Literal["billing", "technical", "general"]` means "one of these three strings, nothing else". That's what makes the output safe to route on.

TRY IT · PYDANTIC VALIDATION — main/m05_pydantic.py

```python
from typing import Literal
from pydantic import BaseModel, ValidationError

class TicketClassification(BaseModel):
    category: Literal["billing", "technical", "general"]
    confidence: float
    reasoning: str

ok = TicketClassification(category="billing", confidence=0.9, reasoning="mentions an invoice")
print(ok.category, ok.confidence)

try:
    TicketClassification(category="refunds", confidence="very sure", reasoning="mentions money")
except ValidationError as e:
    print(e)
```

OUTPUT

```
billing 0.9
2 validation errors for TicketClassification
category
  Input should be 'billing', 'technical' or 'general' [type=literal_error, input_value='refunds', input_type=str]
    For further information visit https://errors.pydantic.dev/2.13/v/literal_error
confidence
  Input should be a valid number, unable to parse string as a number [type=float_parsing, input_value='very sure',
input_type=str]
    For further information visit https://errors.pydantic.dev/2.13/v/float_parsing
```

Read that error carefully. It names the field, says what was expected, and shows what it got. That message matters in a moment, because Instructor sends it back to the model.

How Instructor makes the model fill in your schema

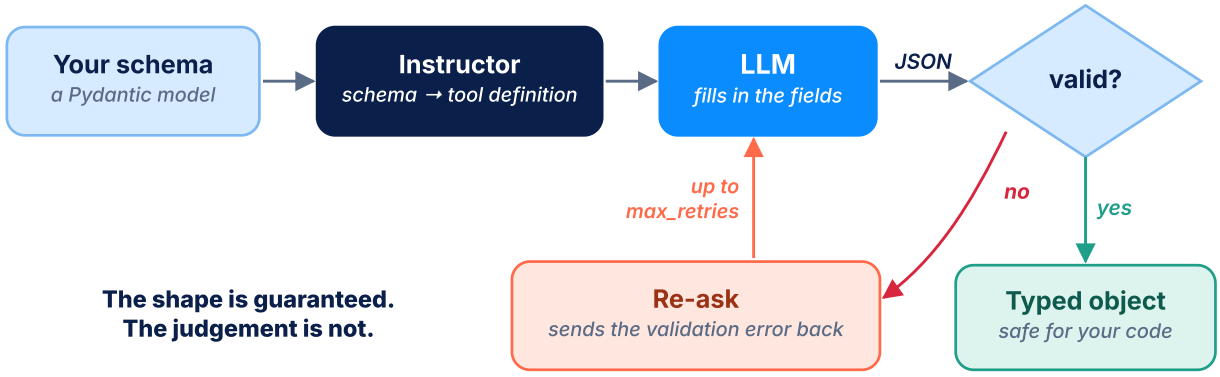


Figure 7.1 `response_model=` *is a constraint, not a suggestion.*

1. Instructor converts your Pydantic class into a **tool definition** (a JSON schema) and tells the model it must call that tool. The allowed `Literal` values become an `enum` in the schema, so the model is told the exact options.
2. The model replies with the tool's arguments as JSON.
3. Instructor validates the JSON against your class. If it passes, you get a real `TicketClassification` object.
4. If it fails, Instructor **re-asks**: it sends the validation error back to the model and tries again, up to `max_retries` times (3 by default in Instructor 1.17).

The cell

NOTEBOOK CELL · STRUCTURED OUTPUT main/m06_structured.py

```python
from pydantic import BaseModel
from typing import Literal

class TicketClassification(BaseModel):
    category: Literal["billing", "technical", "general"]
    confidence: float
    reasoning: str

def classify_with_llm(text: str) -> TicketClassification:
    return client.chat.completions.create(
        model=MODEL,
        response_model=TicketClassification,
        messages=[{"role": "user", "content": f"Classify this support ticket: {text}"}],
    )
```

```
result = classify_with_llm("My server keeps crashing after the last update")
print(result)
```

OUTPUT example: the model's wording and numbers will differ

```
category='technical' confidence=0.92 reasoning='The user reports a server crashing after an update, which is a technical failure.'
```

- The prompt is one line. It never lists the categories, because the schema does that job.
- `classify_with_llm` returns a `TicketClassification`, so `result.category` is guaranteed to be one of the three strings. Your router can use it without a single `if "..." in text`.
- `reasoning` costs a few tokens and pays for itself. It goes straight into the trace (Chapter 11), so you can see *why* a ticket landed where it did.

SHAPE IS GUARANTEED. JUDGEMENT ISN'T.

Structured output guarantees the model can't answer "refunds" or "probably technical?". It does **not** guarantee the model picks the *right* category. A double-charge ticket can still be labelled "technical". That's why we keep a confidence score, log the reasoning, and test the shape instead of exact answers (Chapter 13). And treat `confidence` as a hint, not a probability: it's a number the model wrote, not a measured accuracy.

Your manager asks

"The dashboard crashed. The bot replied 'Sure! Here is your JSON...'"

- Define the output as a schema.
- Validate every response against it.
- Re-ask automatically on a mismatch.

The interviewer asks

"How do you get reliable JSON from an LLM?"

- A Pydantic schema plus `response_model`.
- The library validates and retries.
- Shape is guaranteed; judgement isn't.

The bug: parsing free text with string matching. The fix: a schema.

PITFALLS

- **Using `groq_client` instead of `client`.** Only the Instructor-wrapped `client` understands `response_model=`.
- **Printing the object and parsing the string.** You already have an object: use `result.category`, not `str(result)`.
- **Malformed tool calls on Groq.** If the model botches the tool call itself, Groq often rejects it with a `400 tool_use_failed` error instead of returning bad JSON. That's an API error, so Instructor doesn't re-ask; your retry loop has to (Chapter 9). It happens more with small models.

GO DEEPER · A STRICTER SCHEMA

The notebook's schema accepts `confidence=7.5` and an empty `reasoning`. `Field` adds constraints, and its `description` text is copied into the tool definition, so **the model reads it**. Descriptions are the best place to define what each category means.

PRODUCTION UPGRADE · FIELD CONSTRAINTS deeper/d07_bounded.py

```python
from typing import Literal
from pydantic import BaseModel, Field, ValidationError

class TicketClassification(BaseModel):
    category: Literal["billing", "technical", "general"] = Field(
        description="billing = invoices, charges, refunds; technical = errors, crashes, outages; general = anything else")
    confidence: float = Field(ge=0.0, le=1.0, description="How sure you are, from 0 to 1")
    reasoning: str = Field(min_length=10, description="One sentence explaining the choice")

try:
    TicketClassification(category="billing", confidence=7.5, reasoning="invoice")
except ValidationError as e:
    print(e)
```

OUTPUT

```
2 validation errors for TicketClassification
confidence
  Input should be less than or equal to 1 [type=less_than_equal, input_value=7.5, input_type=float]
    For further information visit https://errors.pydantic.dev/2.13/v/less_than_equal
reasoning
  String should have at least 10 characters [type=string_too_short, input_value='invoice', input_type=str]
    For further information visit https://errors.pydantic.dev/2.13/v/string_too_short
```

Other tools worth knowing: `@field_validator(..., mode="before")` for custom clean-up (for example lower-casing "Billing" before the `Literal` check; the default "after" mode runs too late), `max_retries=` per call, and **modes**. Instructor can use tool calling (the default for Groq), JSON mode, or a provider's native structured outputs. Some providers constrain generation so the output *can't* break the schema; validate-and-retry works everywhere.

CHECKPOINT

1. The model returns `{"category": "Billing", ...}` with a capital B. What happens, step by step?
2. Why is `Literal` a better type for `category` than `str`?
3. Name one thing structured output can't protect you from.

CHAPTER 8 SLIDES 25–26 0:25–0:40 · NOTEBOOK PART 1

The State Machine

Three nodes and a controller. Only one node gets the LLM. Knowing which one, and why, is the whole lesson.

A **state machine** is a system that's always in one known state and moves between states by explicit rules. Ours is built from three ideas:

- **State:** one typed record holding everything known about the ticket so far.
- **Nodes:** plain functions that take the state, do one job, and return the updated state.
- **Controller:** the code that calls the nodes in order. This is the "decides what's next" box from Figure 5.3.

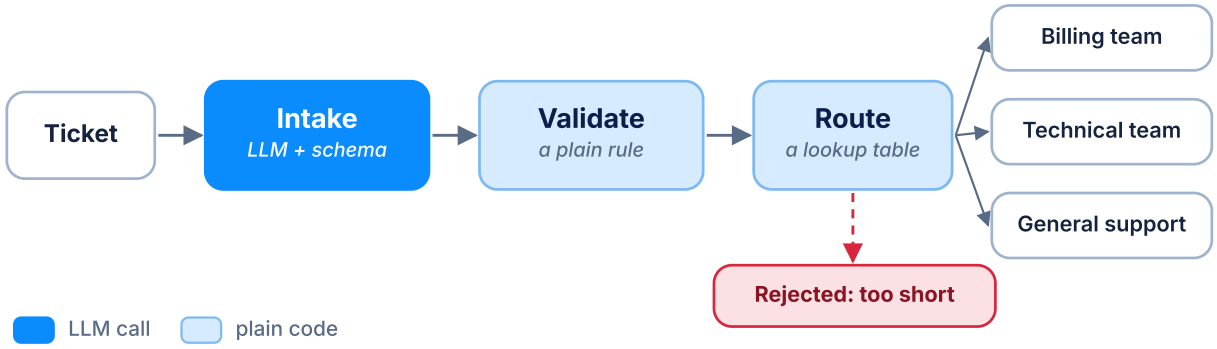


Figure 8.1 *Only intake needs judgement, so only intake calls the model. Validate is a rule; route is a lookup table.*

Cell 1 · The typed state

NOTEBOOK CELL · PART 1, CELL 1 main/m07_state.py

```python
from typing import Literal, Optional
from pydantic import BaseModel

class TicketState(BaseModel):
    ticket_id: str
    text: str
    category: Optional[Literal["billing", "technical", "general", "invalid"]] = None
    confidence: Optional[float] = None
    validated: bool = False
    response: Optional[str] = None
    trace: list[str] = []
```

FIELD	TYPE	WRITTEN BY	MEANING
`ticket_id`, `text`	required strings	you, at creation	The input. Everything else starts empty.
`category`	one of 4 values, or None	intake, validate	The model's label, or `"invalid"` if validation rejects it.
`confidence`	float or None	intake	The model's self-reported confidence.
`validated`	bool	validate	Did the ticket pass the gate?
`response`	str or None	route / handlers	What we tell the customer.
`trace`	list of str	every node	A running log: the agent's memory of what it did.

Cells 2–4 · The nodes

NOTEBOOK CELL · NODE 1, INTAKE (THE LLM NODE) main/m08_intake.py

```python
def intake(state: TicketState) -> TicketState:
    result = classify_with_llm(state.text)
    state.category = result.category
    state.confidence = result.confidence
    state.trace.append(f"intake: classified as '{result.category}' (confidence={result.confidence:.2f}):
{result.reasoning}")
    return state
```

Same shape as every node: state in, state out. The classification is now a real model judgement with confidence and reasoning attached, not a hard-coded keyword check.

NOTEBOOK CELL · NODE 2, VALIDATE (A PLAIN RULE) main/m09_validate.py

```python
def validate(state: TicketState) -> TicketState:
    if len(state.text.strip()) < 5:
        state.category = "invalid"
        state.validated = False
        state.trace.append("validate: rejected, ticket text too short")
    else:
        state.validated = True
        state.trace.append("validate: passed")
    return state
```

Deliberately no LLM. "Is the text at least 5 characters?" is a rule, and rules are free, instant, and never wrong about themselves.

NOTEBOOK CELL · NODE 3, THE HANDLERS PLUS ROUTING main/m10_route.py

```python
def handle_billing(state: TicketState) -> TicketState:
    state.response = "Routed to Billing team. A refund review will follow within 2 business days."
    state.trace.append("respond: handled by billing")
    return state

def handle_technical(state: TicketState) -> TicketState:
    state.response = "Routed to Technical team. Logs are being pulled for your account."
    state.trace.append("respond: handled by technical")
    return state

def handle_general(state: TicketState) -> TicketState:
    state.response = "Routed to General support. Expect a reply within 24 hours."
    state.trace.append("respond: handled by general")
    return state

def route(state: TicketState) -> TicketState:
    if not state.validated:
        state.response = "Ticket rejected: not enough information."
        state.trace.append("route: rejected at gate")
        return state
    handlers = {"billing": handle_billing, "technical": handle_technical, "general": handle_general}
    return handlers[state.category](state)
```

`route` is the **rails** from Figure 5.5. A rejected ticket stops at the gate. Otherwise a **dispatch table** (a dictionary from category to function) picks the handler. Adding a team later means adding one function and one dictionary entry, not another `elif`.

Cell 5 · The controller

NOTEBOOK CELL · PART 1, CELL 5 main/m11_controller.py

```python
def run_agent(ticket_id: str, text: str) -> TicketState:
    state = TicketState(ticket_id=ticket_id, text=text)
    state = intake(state)     # observe + think, now backed by a real model call
    state = validate(state)   # think, deterministic
    state = route(state)      # act
    return state

result = run_agent("T-001", "My invoice #4521 charged me twice.")
print(result.response)
print(result.trace)
```

OUTPUT

example: the model's wording and numbers will differ

```
Routed to Billing team. A refund review will follow within 2 business days.
["intake: classified as 'billing' (confidence=0.95): The ticket is about an invoice and a duplicate charge, a billing
issue.", 'validate: passed', 'respond: handled by billing']
```

Here's how the state fills in as the ticket moves through:

AFTER	CATEGORY	CONFIDENCE	VALIDATED	RESPONSE	TRACE
creation	None	None	False	None	0 lines
INTAKE	billing	0.95	False	None	1 line
VALIDATE	billing	0.95	True	None	2 lines
ROUTE	billing	0.95	True	"Routed to Billing team..."	3 lines

Your manager asks

"Our LLM bill tripled this month. Why?"

- Find every step that calls the model.
- Turn rule-shaped steps into plain code.
- Keep the LLM for judgement only.

The interviewer asks

"Which nodes in your agent call the model, and why only those?"

- Only intake: it needs judgement on messy text.
- Validate and route are plain rules.
- Fewer calls: cheaper, faster, testable.

PITFALL · ASSIGNMENTS AREN'T VALIDATED

Pydantic checks values when the object is *created*, not when you later assign a field. `state.category = "refunds"` is accepted silently, and the error only surfaces later in `route`. The fix is one line on the class, `model_config = ConfigDict(validate_assignment=True)`, after `from pydantic import ConfigDict`. (Mutable defaults like `trace: list[str] = []` are safe in Pydantic, which copies them per object, unlike a plain Python class.)

GO DEEPER · WHY DOES THE LLM RUN BEFORE VALIDATE?

Look at the controller order: `intake` runs first, so a ticket that says only "hi" still costs a model call before validation throws it away. Swap the order and junk tickets cost nothing:

PRODUCTION UPGRADE · VALIDATE FIRST

deeper/d08_validate_first.py

```python
def run_agent_cheap(ticket_id: str, text: str) -> TicketState:
    state = TicketState(ticket_id=ticket_id, text=text)
    state = validate(state)          # free: a plain rule
    if not state.validated:
        return route(state)          # rejected without paying for a model call
    state = intake(state)            # only real tickets reach the LLM
    return route(state)

result = run_agent_cheap("T-006", "hi")
print(result.response)
print(result.trace)
```

OUTPUT

```
Ticket rejected: not enough information.
['validate: rejected, ticket text too short', 'route: rejected at gate']
```

No `intake` line in the trace means no model call. At thousands of tickets a day, gate-before-model ordering is real money.

EXERCISE · TRACE IT BY HAND

Before running anything, write down the trace lines and final `response` for each ticket. Then run `run_agent` and compare. Answers are in the Answer Key.

1. `"hi"`
2. `"Please refund my last payment"`
3. `"The login page shows error 500"`

CHAPTER 9 SLIDE 27 0:40–0:50 · LIVE BREAK

Errors, Retries & Fallback

Every external call fails eventually. Plan for the call that fails, because it will, and make the system degrade gracefully instead of crashing.

What breaks in production

Your agent depends on a network call to someone else's servers. Here's what goes wrong, and whether trying again can help:

FAILURE	YOU SEE	CAUSE	RETRY?
Rate limit	`429`	Too many requests or tokens per minute (very common on free tiers)	**Yes, after a wait**
Server error / overload	`500` , `502` , `503`	The provider is having a bad minute	**Yes, after a wait**
Timeout / connection	`APITimeoutError`	Network blip, slow response, Wi-Fi in a lecture hall	**Yes**
Bad output	`ValidationError` , or Groq's `400 tool_use_failed`	The model broke the schema, or botched the tool call itself	**Instructor re-asks on the first; your loop must retry the second**
Bad key	`401`	Wrong, revoked or missing API key	**No: fix the key**
Bad request	`400` , `404`	Malformed request, retired model name	**No: fix the code**

The rule: **retry what's temporary, fail fast on what's permanent.** Waiting won't fix a typo in your API key.

Exponential backoff, then a fallback

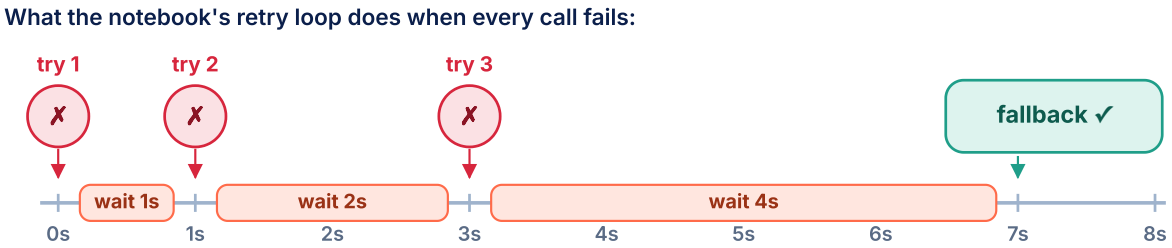


Figure 9.1 *Wait 1s, then 2s, then 4s. If every try fails, answer with something simpler instead of crashing.*

Why wait longer each time? If a service is overloaded, hammering it with instant retries makes the overload worse, for you and everyone else. Doubling the wait (`2 ** (attempt - 1)`) backs off quickly while still recovering fast from a one-off blip.

Why a fallback? Because "crash with a stack trace" is the worst possible behaviour for a support system. Our fallback is the old keyword matcher: crude, but always available, instant and free. That's *graceful degradation*: worse answers beat no answers.

NOTEBOOK CELL · RETRY, BACKOFF, FALLBACK main/m12_retry.py

```python
import time

def classify_with_retry(text: str, max_attempts: int = 3) -> TicketClassification:
    for attempt in range(1, max_attempts + 1):
        try:
            return classify_with_llm(text)
        except Exception as e:
            wait = 2 ** (attempt - 1)  # exponential backoff: 1s, 2s, 4s
            print(f"attempt {attempt} failed ({e}), retrying in {wait}s...")
            time.sleep(wait)
    print(f"all {max_attempts} attempts failed, falling back to keyword matching")
    return fallback_keyword_classify(text)

def fallback_keyword_classify(text: str) -> TicketClassification:
    t = text.lower()
    if "invoice" in t or "charge" in t:
        cat = "billing"
    elif "error" in t or "bug" in t or "crash" in t:
        cat = "technical"
    else:
        cat = "general"
    return TicketClassification(category=cat, confidence=0.5, reasoning="fallback: keyword match, LLM
unavailable")

# update intake to use the safe version
def intake(state: TicketState) -> TicketState:
    result = classify_with_retry(state.text)
    state.category = result.category
    state.confidence = result.confidence
    state.trace.append(f"intake: classified as '{result.category}' (confidence={result.confidence:.2f}):
{result.reasoning}")
    return state
```

- `classify_with_retry` wraps the LLM call in a loop. Success returns immediately; failure prints, sleeps and tries again.
- The fallback returns the **same type** (`TicketClassification`), so nothing downstream has to know which path produced it.

- It marks itself honestly: `confidence=0.5` and a reasoning that starts with `"fallback:"`. Anyone reading the trace can see the LLM was unavailable.
- `intake` is redefined to call the safe version. Same name, same shape, so the controller doesn't change at all.

The live break

In the session we break it on purpose. Here's a cell that does the same thing safely: swap in a client with a garbage key, run a ticket, then put the real client back.

NOTEBOOK CELL · BREAK IT ON PURPOSE main/m13_break.py

```python
# Break it on purpose: swap in a client with a garbage key
client = instructor.from_groq(Groq(api_key="gsk_not_a_real_key"))

result = run_agent("T-002", "My invoice #4521 charged me twice.")
print(result.response)
print(result.trace[0])

# Put the real client back before moving on!
client = instructor.from_groq(groq_client)
```

OUTPUT real output, captured with a fake key

```
attempt 1 failed (Error code: 401 - {'error': {'message': 'Invalid API Key', 'type': 'invalid_request_error', 'code':
'invalid_api_key'}}), retrying in 1s...
attempt 2 failed (Error code: 401 - {'error': {'message': 'Invalid API Key', 'type': 'invalid_request_error', 'code':
'invalid_api_key'}}), retrying in 2s...
attempt 3 failed (Error code: 401 - {'error': {'message': 'Invalid API Key', 'type': 'invalid_request_error', 'code':
'invalid_api_key'}}), retrying in 4s...
all 3 attempts failed, falling back to keyword matching
Routed to Billing team. A refund review will follow within 2 business days.
intake: classified as 'billing' (confidence=0.50): fallback: keyword match, LLM unavailable
```

You'll also see two extra lines per attempt starting `API call failed on attempt 1` and `Max retries exceeded`. That's Instructor logging the same error, and it's harmless. The important part: the customer still got routed to Billing. The trace explains why the confidence is 0.50.

NOTICE WHAT JUST HAPPENED

The failure was a **401**, a bad key. Our loop retried it three times and waited 7 seconds before falling back, and none of that could ever have worked. The notebook version is deliberately simple; the Go deeper box below fixes it. There's also a hidden layer: the Groq SDK **already retries** 429s, 5xx errors and connection failures twice on its own before your loop even sees an error. During an outage, one ticket can make nine requests (3 tries × 3 SDK attempts), and more if the model also keeps failing validation, since Instructor re-asks up to 3 times per call. Know your retry layers.

Your manager asks

"The provider had an outage during the demo and everything crashed."

- Retry temporary failures with backoff.
- Fall back to something simpler that can't fail.
- Log it, so we see how often it happens.

The interviewer asks

"What happens when your LLM provider goes down?"

- Bounded retries with exponential backoff.
- A deterministic fallback, clearly marked.
- Degrade gracefully; never crash the request.

GO DEEPER · RETRY ONLY WHAT'S RETRYABLE

Instructor wraps the SDK's exception, so the real cause is in `e.__cause__`. Check it, give up immediately on permanent errors (with one Groq-specific exception: a `400 tool_use_failed` is worth asking again), skip the pointless sleep after the last try, and add a little random **jitter** so a thousand clients that failed together don't all retry at the same instant.

PRODUCTION UPGRADE · SMARTER RETRY deeper/d09_smart_retry.py

```python
import random
import time
import groq

# Retrying can't fix these: the request itself is wrong
NOT_RETRYABLE = (groq.AuthenticationError, groq.PermissionDeniedError,
                 groq.BadRequestError, groq.NotFoundError)

def is_retryable(err):
    # exception: Groq rejects a malformed tool call with a 400 "tool_use_failed"; asking again usually works
    if isinstance(err, groq.BadRequestError) and "tool_use_failed" in str(err):
        return True
    return not isinstance(err, NOT_RETRYABLE)

def classify_with_retry(text: str, max_attempts: int = 3) -> TicketClassification:
    for attempt in range(1, max_attempts + 1):
        try:
            return classify_with_llm(text)
        except Exception as e:
            root = e.__cause__ or e                # Instructor wraps the SDK's error
            if not is_retryable(root):
                print(f"{type(root).__name__} is not retryable, falling back now")
                break
            if attempt == max_attempts:
                break                              # no point sleeping after the last try
            wait = 2 ** (attempt - 1) + random.uniform(0, 0.5)   # backoff + jitter
            print(f"attempt {attempt} failed ({type(root).__name__}), retrying in {wait:.1f}s...")
            time.sleep(wait)
    return fallback_keyword_classify(text)

client = instructor.from_groq(Groq(api_key="gsk_not_a_real_key"))
start = time.time()
print(classify_with_retry("My invoice #4521 charged me twice."))
print(f"took {time.time() - start:.1f}s instead of 7s")
client = instructor.from_groq(groq_client)
```

OUTPUT real output, captured with a fake key

```
AuthenticationError is not retryable, falling back now
category='billing' confidence=0.5 reasoning='fallback: keyword match, LLM unavailable'
took 0.1s instead of 7s
```

Also worth knowing: set a **timeout** (`Groq(timeout=10)`) so a hung call can't stall a ticket for a minute; the **tenacity** library implements all of this as a decorator; and a **circuit breaker** stops calling a provider entirely for a while after repeated failures.

CHECKPOINT

1. Your agent gets a `429`. Should it retry? Should it retry immediately?
2. Why does the fallback return a `TicketClassification` rather than a plain string?
3. The fallback sees "Please refund my last payment". What category does it return, and what does that tell you about keyword rules?

CHAPTER 10 SLIDE 29 0:50–1:05 · NOTEBOOK PART 2

Branching & the Reducer

Two independent checks, merged back into one shared state. Whether it's two checks or twenty, the pattern is the same.

Real triage needs more than a category. Two more questions matter, and neither depends on the other or on the LLM:

- **Compliance:** does the ticket mention legal action? Then it must go to a human reviewer, whatever the category.
- **Priority:** does it sound urgent? Then the team should see it first.

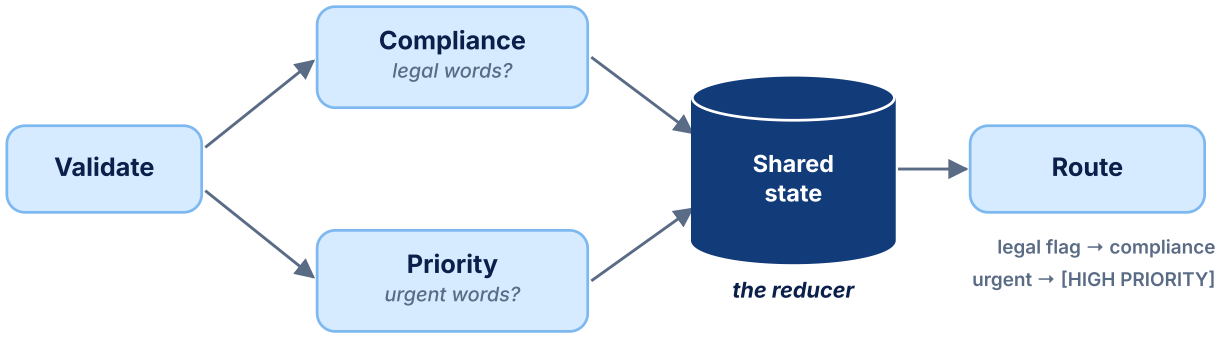


Figure 10.1 *Branches run independently and write to different fields of one shared state. The merged state decides the route.*

The checks

NOTEBOOK CELL · NEW STATE FIELDS PLUS TWO CHECKS main/m14_branches.py

```python
class TicketState(BaseModel):
    ticket_id: str
    text: str
    category: Optional[Literal["billing", "technical", "general", "invalid"]] = None
    confidence: Optional[float] = None
    validated: bool = False
    compliance_flag: Optional[bool] = None    # new
    priority: Optional[Literal["normal", "high"]] = None    # new
    response: Optional[str] = None
    trace: list[str] = []

def check_compliance(state: TicketState) -> TicketState:
    banned = ["lawsuit", "lawyer", "legal action"]
    state.compliance_flag = any(word in state.text.lower() for word in banned)
    state.trace.append(f"compliance_check: flag={state.compliance_flag}")
    return state

def check_priority(state: TicketState) -> TicketState:
```

```
    urgent_words = ["immediately", "urgent", "down", "outage", "asap"]
    state.priority = "high" if any(w in state.text.lower() for w in urgent_words) else "normal"
    state.trace.append(f"priority_check: priority={state.priority}")
    return state
```

`TicketState` gets two new optional fields. Both checks are **deterministic on purpose**: no LLM call and no retry, because they don't need one. Not every node should hit the model; that's a cost and latency decision as much as a design one.

The reducer, and routing on the merged state

NOTEBOOK CELL · MERGE AND ROUTE

main/m15_reducer.py

```
def run_branches(state: TicketState) -> TicketState:
    state = check_compliance(state)
    state = check_priority(state)
    return state  # both branches wrote to the same state object, that merge IS the reducer

def route_v2(state: TicketState) -> TicketState:
    if not state.validated:
        state.response = "Ticket rejected: not enough information."
        return state
    if state.compliance_flag:
        state.response = "Escalated to Legal/Compliance review."
        state.trace.append("route: compliance override")
        return state
    prefix = "[HIGH PRIORITY] " if state.priority == "high" else ""
    handlers = {"billing": handle_billing, "technical": handle_technical, "general": handle_general}
    state = handlers[state.category](state)
    state.response = prefix + state.response
    return state

def run_agent_v2(ticket_id: str, text: str) -> TicketState:
    state = TicketState(ticket_id=ticket_id, text=text)
    state = intake(state)         # structured output + retry
    state = validate(state)
    state = run_branches(state)   # the two parallel checks, merged
    state = route_v2(state)       # the reducer's output decides routing
    return state

print(run_agent_v2("T-003", "The app is down and I need it fixed immediately").response)
print(run_agent_v2("T-004", "My lawyer will hear about this double charge").response)
```

OUTPUT

example: the model's wording and numbers will differ

```
[HIGH PRIORITY] Routed to Technical team. Logs are being pulled for your account.
Escalated to Legal/Compliance review.
```

A **reducer** is whatever combines several branches' outputs into one state. Here there's no special function: both checks write to the *same* state object, each to its own field, so the merge is automatic. `route_v2` then reads the merged result with a clear order of precedence:

ORDER	CONDITION	OUTCOME
1	not validated	Rejected, and nothing else matters
2	compliance flag	Escalated to Legal/Compliance, which overrides the category
3	priority high	`[HIGH PRIORITY]` prefix on the normal team response
4	otherwise	The normal handler for the category

PITFALL · SUBSTRING MATCHING IS SNEAKY

`"down" in text` is also true for "down**load**" and "count**down**". So "I can't download my invoice" gets flagged **high priority**. Use word boundaries (`re.search(r"\bdown\b", text)`) or, better, move fuzzy judgements like urgency into the LLM's schema, where they belong.

GO DEEPER · REAL PARALLEL BRANCHES NEED A REAL REDUCER

In our hand-built version the "branches" run one after the other. In LangGraph you can run them **in the same step**, by drawing two edges out of one node. But if two parallel nodes write the *same* key, LangGraph can't know how to combine them and stops with:

`InvalidUpdateError: At key 'trace': Can receive only one value per step. Use an Annotated key to handle multiple values.`

The fix is to declare a reducer on that key. `Annotated[list, operator.add]` means "when several updates arrive, concatenate them":

PRODUCTION UPGRADE · PARALLEL FAN-OUT WITH A REDUCER deeper/d10_fanout.py

```python
import operator
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, START, END

class FanState(TypedDict):
    text: str
    compliance_flag: bool
    priority: str
    trace: Annotated[list, operator.add]   # the reducer: concatenate lists from parallel branches

def fan_compliance(state):
    flag = any(w in state["text"].lower() for w in ["lawsuit", "lawyer", "legal action"])
    return {"compliance_flag": flag, "trace": [f"compliance: flag={flag}"]}

def fan_priority(state):
    level = "high" if any(w in state["text"].lower() for w in ["urgent", "down", "asap"]) else "normal"
    return {"priority": level, "trace": [f"priority: {level}"]}

def merge(state):
    return {"trace": [f"merge: flag={state['compliance_flag']}, priority={state['priority']}"]}

fan = StateGraph(FanState)
fan.add_node("compliance", fan_compliance)
fan.add_node("priority", fan_priority)
fan.add_node("merge", merge)
fan.add_edge(START, "compliance")                    # two edges out of START:
fan.add_edge(START, "priority")                      # both branches run in the same step
fan.add_edge(["compliance", "priority"], "merge")    # wait for BOTH before merging
fan.add_edge("merge", END)

print(fan.compile().invoke({"text": "Site is down and my lawyer is calling", "trace": []}))
```

OUTPUT

```
{'text': 'Site is down and my lawyer is calling', 'compliance_flag': True, 'priority': 'high', 'trace':
['compliance: flag=True', 'priority: high', 'merge: flag=True, priority=high']}
```

Note the nodes return **only the keys they change**, and `add_edge(["compliance", "priority"], "merge")` waits for both branches before merging.

CHECKPOINT

1. A ticket says "Our site is down and my lawyer is calling." What does `route_v2` return, and why?
2. Why are the compliance and priority checks not LLM calls?
3. In your own words: what is a reducer?

CHAPTER 11 · SLIDE 30 · 0:50–1:05 · NOTEBOOK PART 2

Observability

The first question you'll be asked about any agent in production is "why did it do that?" A trace is how you answer with evidence instead of a guess.

So far, our trace is a list of human-readable strings. That's a start, but you can't search, filter or chart it. Production systems log **structured** entries for every step: which node ran, what came in, what came out, and how long it took. We add that without touching a single node, using a **decorator**.

Decorators in one minute

In Python, functions are values: you can pass them around and wrap them. A decorator is a function that takes your function and returns a new one that does something extra *around* the original. `traced_node("intake")(intake)` means "give me a version of `intake` that also times itself and logs the result".

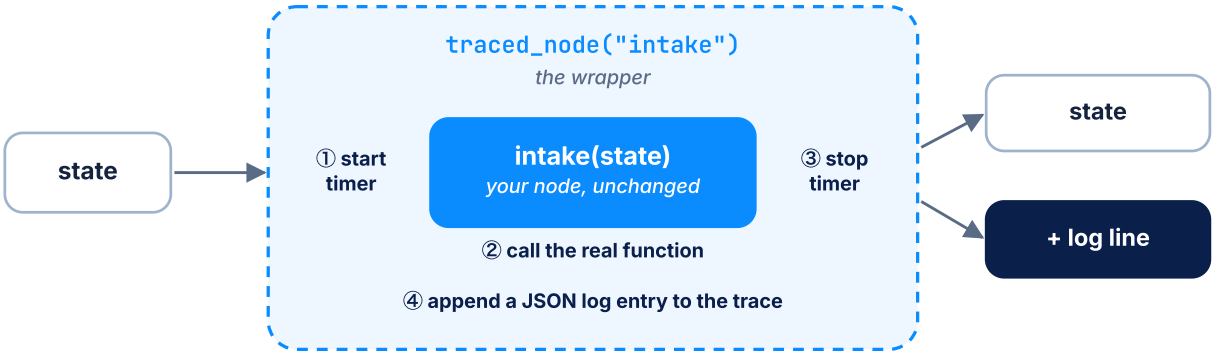


Figure 11.1 *The wrapper adds timing and logging; your node runs unchanged in the middle.*

NOTEBOOK CELL · STRUCTURED TRACING — main/m16_traced.py

```python
import time as time_module
import json
import functools

def traced_node(node_name):
    def decorator(fn):
        @functools.wraps(fn)
        def wrapper(state):
            start = time_module.time()
            result = fn(state)
            duration_ms = round((time_module.time() - start) * 1000, 1)
            log_entry = {
                "node": node_name,
                "duration_ms": duration_ms,
                "output_category": result.category,
                "confidence": result.confidence,
```

```
            }
            result.trace.append(json.dumps(log_entry))
            return result
        return wrapper
    return decorator

intake = traced_node("intake")(intake)
validate = traced_node("validate")(validate)

result = run_agent_v2("T-005", "My server keeps crashing, this is urgent")
for line in result.trace:
    print(line)
```

OUTPUT example: the model's wording and numbers will differ

```
intake: classified as 'technical' (confidence=0.93): Server crashes are a technical issue; the user also flags
urgency.
{"node": "intake", "duration_ms": 609.6, "output_category": "technical", "confidence": 0.93}
validate: passed
{"node": "validate", "duration_ms": 0.0, "output_category": "technical", "confidence": 0.93}
compliance_check: flag=False
priority_check: priority=high
respond: handled by technical
```

- **Three layers of functions**: `traced_node(name)` takes the node's name and returns `decorator`, which takes the function and returns `wrapper`, which is what actually runs. The extra layer exists so you can pass the name in.
- `@functools.wraps(fn)` copies the original function's name and docstring onto the wrapper, so tracebacks still say `intake`, not `wrapper`.
- `json.dumps` makes each entry a machine-readable line: you can load it into a dataframe or a log system and filter by node, duration or category.
- Rebinding `intake = traced_node("intake")(intake)` works because `run_agent_v2` looks up `intake` by name each time it runs.

RUN THE TRACING CELL ONCE

Re-running it wraps the *already wrapped* `intake` and `validate` again, and every log line appears twice. If that happens, re-run the cells that define them (the retry cell for `intake`, node 2 for `validate`), then this one.

Reading a trace

NODE	TIME	WHAT IT TELLS YOU
`intake`	~600 ms	Almost all the latency is the LLM call. Optimise here, not elsewhere.
`validate`	0.0 ms	Plain code is effectively free.
`priority_check`	(string entry)	`high`, because "urgent" is in the text. Explains the `[HIGH PRIORITY]` prefix.
`respond`	(string entry)	Which handler finally ran.

Note: the decorator logs the *state's* category after the node runs, so the `validate` entry repeats the category intake set. That's useful (you see the state at each step), but it isn't validate's own output.

FROM TOY TO PRODUCTION

This is a toy version of what **LangSmith**, **Langfuse** and **OpenTelemetry**-based tools do automatically: capture every node's input, output and timing for every run, then give you a UI to search across thousands of runs. The concept is identical; they just do it at scale with a dashboard instead of `print`.

The interviewer asks

"How would you debug an agent that made the wrong call in production?"

- **"I'd look at the trace."** Find the run by its ID and read node by node: which node made the decision, what input it saw, what it output.
- Reproduce it: run the same input through that node alone.
- Fix the node (prompt, schema, rule or ordering), then add the input as a **regression test** so it can't silently come back.

GO DEEPER · A TRACE YOU CAN ACTUALLY DEBUG WITH

The notebook version has three gaps. It doesn't record inputs. It doesn't say which run an entry belongs to. And if a node raises an exception, no entry is written, which is exactly when you need one most. This version fixes all three, uses `time.perf_counter()` (a clock that never jumps), and re-raises errors rather than swallowing them:

PRODUCTION UPGRADE · TRACING WITH ERRORS AND RUN IDS deeper/d11_traced_v2.py

```python
def traced_node_v2(node_name):
    def decorator(fn):
        @functools.wraps(fn)
        def wrapper(state):
            start = time_module.perf_counter()        # monotonic: immune to clock changes
            entry = {"run_id": state.ticket_id, "node": node_name, "input_chars": len(state.text)}
            try:
                result = fn(state)
                entry.update(status="ok", category=result.category, confidence=result.confidence)
                return result
            except Exception as e:
                entry.update(status="error", error=f"{type(e).__name__}: {e}")
                raise                                  # log it, but never swallow it
            finally:
                entry["duration_ms"] = round((time_module.perf_counter() - start) * 1000, 1)
                print(json.dumps(entry))               # production: send to a logger, not print
        return wrapper
    return decorator

@traced_node_v2("explode")
def explode(state):
    raise ValueError("simulated bug in a node")

try:
    explode(TicketState(ticket_id="T-007", text="anything at all"))
except ValueError:
    pass
```

OUTPUT

```
{"run_id": "T-007", "node": "explode", "input_chars": 15, "status": "error", "error": "ValueError: simulated bug in a node", "duration_ms": 0.0}
```

CHECKPOINT

1. Why are JSON log lines more useful than f-strings at scale?
2. Your trace shows `intake` took 4,000 ms on one ticket. Name two possible causes.
3. Why should a tracing wrapper re-raise exceptions instead of catching them?

CHAPTER 12 SLIDE 31 1:05–1:20 · NOTEBOOK PART 3

The LangGraph Rebuild

LangGraph is a way to formalise the state machine you just built. Same schema, same retry logic, same checks, now wired with `add_node` and `add_edge` instead of hand-written calls.

Why frameworks exist

Your hand-built controller is five lines and works perfectly. So why use a framework? Because of what you'd have to build next: conditional routing between nodes, branches that really run in parallel, merging their outputs safely, pausing for a human and resuming later, saving state so a crash doesn't lose work, streaming progress to a UI, and drawing the graph for your teammates. LangGraph gives you all of that for the price of describing your machine in its vocabulary.

The whiteboard model

State
The schema: what we track. A `TypedDict` here, the same fields as `TicketState`.

Nodes
The circles: plain Python functions you already trust. State in, update out.

Edges
The lines: where to go next. Fixed (`add_edge`) or chosen by a function (`add_conditional_edges`).

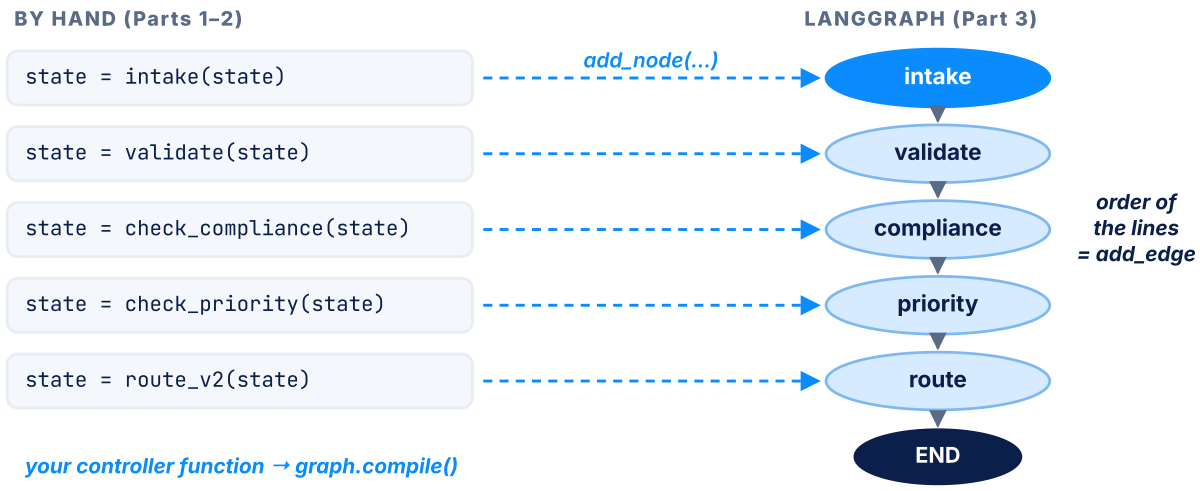


Figure 12.1 *Every node call in your controller becomes `add_node`; every "then" becomes `add_edge`; the controller itself becomes `compile()`.*

The cell

NOTEBOOK CELL · PART 3, THE LANGGRAPH REBUILD — main/m17_langgraph.py

```python
from langgraph.graph import StateGraph, END
from typing import TypedDict

class GraphState(TypedDict):
    ticket_id: str
    text: str
    category: str
    confidence: float
```

```
    validated: bool
    compliance_flag: bool
    priority: str
    response: str
    trace: list

def lg_intake(state: GraphState) -> GraphState:
    result = classify_with_retry(state["text"])   # structured output + retry, unchanged
    trace = state.get("trace", []) + [f"intake: {result.category} (confidence={result.confidence:.2f})"]
    return {**state, "category": result.category, "confidence": result.confidence, "trace": trace}

def lg_validate(state: GraphState) -> GraphState:
    return {**state, "validated": len(state["text"].strip()) >= 5}

def lg_compliance(state: GraphState) -> GraphState:
    banned = ["lawsuit", "lawyer", "legal action"]
    return {**state, "compliance_flag": any(w in state["text"].lower() for w in banned)}

def lg_priority(state: GraphState) -> GraphState:
    urgent = ["immediately", "urgent", "down", "outage", "asap"]
    return {**state, "priority": "high" if any(w in state["text"].lower() for w in urgent) else "normal"}

def lg_route(state: GraphState) -> GraphState:
    if not state["validated"]:
        return {**state, "response": "Ticket rejected: not enough information."}
    if state["compliance_flag"]:
        return {**state, "response": "Escalated to Legal/Compliance review."}
    prefix = "[HIGH PRIORITY] " if state["priority"] == "high" else ""
    replies = {"billing": "Routed to Billing.", "technical": "Routed to Technical.", "general": "Routed to
General."}
    return {**state, "response": prefix + replies[state["category"]]}

graph = StateGraph(GraphState)
graph.add_node("intake", lg_intake)
graph.add_node("validate", lg_validate)
graph.add_node("compliance", lg_compliance)
graph.add_node("priority", lg_priority)
graph.add_node("route", lg_route)

graph.set_entry_point("intake")
graph.add_edge("intake", "validate")
graph.add_edge("validate", "compliance")
graph.add_edge("compliance", "priority")
graph.add_edge("priority", "route")
graph.add_edge("route", END)

app = graph.compile()
result = app.invoke({"ticket_id": "T-010", "text": "Our production server is down, this is urgent", "trace": []})
print(result["response"])
print(result["trace"])
```

OUTPUT example: the model's wording and numbers will differ

```
[HIGH PRIORITY] Routed to Technical.
['intake: technical (confidence=0.94)']
```

Line by line

- `GraphState(TypedDict)`: a dictionary with declared keys. LangGraph passes state between nodes as a plain dict, so nodes use `state["text"]` instead of `state.text`.
- **Each `lg_` node** returns `{**state, "key": value}`: a copy of the whole state with one or two keys changed. `lg_intake` still calls `classify_with_retry`, so structured output, retries and the fallback all carry over untouched.

- `add_node("intake", lg_intake)` registers a function under a name. Edges refer to the *name*, not the function.
- `set_entry_point("intake")` says where every run starts (it draws an edge from the special `START` node). `END` is where it stops.
- `compile()` checks the wiring (for example that every edge points at a node that exists) and returns `app`, a runnable you call with `invoke(initial_state)`. The result is the final state dict.

Notice the trace has one line. In this version only `lg_intake` appends to it; the other nodes just compute. That's fine for now, and adding trace entries to a node is part of the lab.

Seeing inside the graph

Two habits that will save you hours: **draw** the graph, and **watch which nodes actually ran**.

NOTEBOOK CELL · INSPECT THE GRAPH — main/m18_inspect.py

```
# 1. Print the wiring as Mermaid text (paste it into mermaid.live to see the picture)
print(app.get_graph().draw_mermaid())

# 2. Watch which nodes ACTUALLY ran, in order
for step in app.stream({"ticket_id": "T-011", "text": "I was charged twice this month", "trace": []},
                       stream_mode="updates"):
    print(list(step))
```

OUTPUT — Mermaid header lines trimmed

```
        __start__ --> intake;
        compliance --> priority;
        intake --> validate;
        priority --> route;
        validate --> compliance;
        route --> __end__;
['intake']
['validate']
['compliance']
['priority']
['route']
```

`draw_mermaid()` prints the wiring as text; paste it into **mermaid.live** for a picture, or in Colab run `Image(app.get_graph().draw_mermaid_png())` after `from IPython.display import Image`. `stream(..., stream_mode="updates")` yields one item per node as it runs: a live, built-in trace of the path a ticket took.

THE "AHA"

Every node here is a function you already trust: structured output, retry and fallback, all carried over. LangGraph didn't replace your engineering. It gave you `add_node`, `add_edge` and `compile()` instead of wiring the calls together yourself. That's the entire value of a framework, and now you can say that in an interview instead of just naming one.

PITFALLS

- **Edited a node but nothing changed?** The graph stores the function object you passed to `add_node`. After editing a node function, re-run the cell that builds and compiles the graph.
- **Forgot `compile()`**: a `StateGraph` is a blueprint and has no `invoke`. Only the compiled `app` does.
- **Forgot the `return`**: LangGraph treats `None` as "no update". Reassigned keys silently vanish, with no error. (In-place edits like `.append()` can appear to stick, which makes it even more confusing.) Never mutate the input; return the update.
- **Mixing up names**: edges use the node's *name* (`"intake"`), not the function's (`lg_intake`).

GO DEEPER · HOW A PRODUCTION TEAM WOULD WRITE THIS GRAPH

The notebook graph is deliberately a line-by-line port. A production version would change four things:

1. **Partial updates**: nodes return only the keys they change, not `{**state, ...}`. LangGraph merges them, and parallel branches can't overwrite each other.
2. **A reducer on `trace`** (`Annotated[list[str], operator.add]`), so every node just returns its own new lines.
3. **Conditional edges for routing**: a small function reads the state and returns the *name* of the next node, so each team gets its own node and the path shows up in the graph and in `stream()`.
4. **Validate before intake** and **fan out** the two checks in parallel.

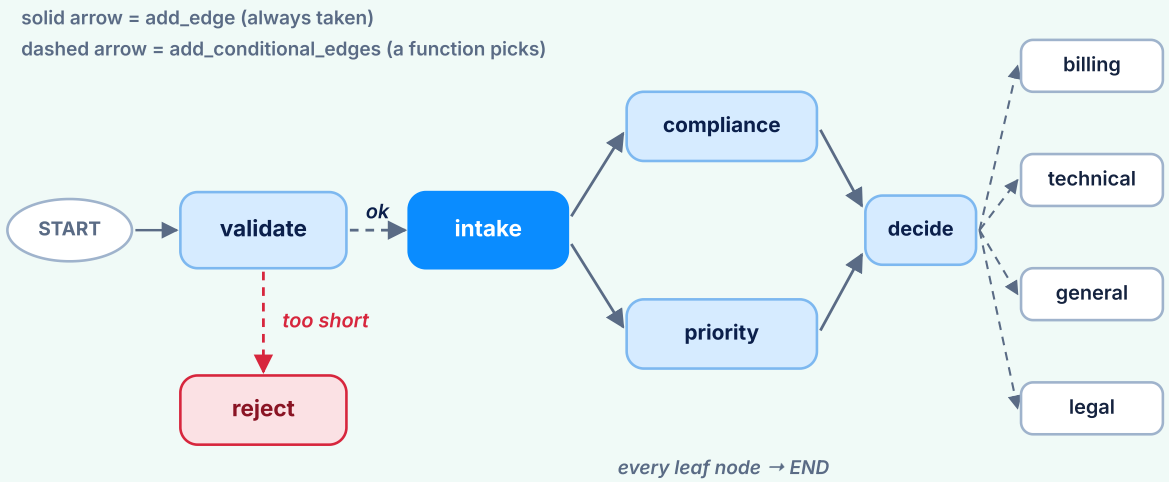


Figure 12.2 *The production-style graph built by the listing below.*

`add_conditional_edges` takes a routing function plus either a **list** of possible next nodes (when the function returns node names, as here) or a **dict** mapping its return values to node names (as in the lab). This cell runs on top of your notebook; the complete production version, with every Go deeper upgrade in one runnable script, ships with this guide as `triage_agent.py`.

PRODUCTION UPGRADE · CONDITIONAL EDGES, REDUCER, FAN-OUT deeper/d12_langgraph_v2.py

```
import operator
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, START, END

class TriageState(TypedDict, total=False):
    ticket_id: str
    text: str
    category: str
    confidence: float
```

```
    validated: bool
    compliance_flag: bool
    priority: str
    response: str
    trace: Annotated[list[str], operator.add]     # every node appends; LangGraph merges

# Nodes return ONLY the keys they change (partial updates), never {**state, ...}
def v2_validate(state):
    ok = len(state["text"].strip()) >= 5
    return {"validated": ok, "trace": [f"validate: {'passed' if ok else 'rejected'}"]}

def v2_reject(state):
    return {"response": "Ticket rejected: not enough information.", "trace": ["reject: too short"]}

def v2_intake(state):
    result = classify_with_retry(state["text"])
    return {"category": result.category, "confidence": result.confidence,
            "trace": [f"intake: {result.category} ({result.confidence:.2f})"]}

def v2_compliance(state):
    flag = any(w in state["text"].lower() for w in ["lawsuit", "lawyer", "legal action"])
    return {"compliance_flag": flag, "trace": [f"compliance: {flag}"]}

def v2_priority(state):
    urgent = ["immediately", "urgent", "down", "outage", "asap"]
    level = "high" if any(w in state["text"].lower() for w in urgent) else "normal"
    return {"priority": level, "trace": [f"priority: {level}"]}

def make_handler(team, reply, use_prefix=True):
    def handler(state):
        prefix = "[HIGH PRIORITY] " if use_prefix and state.get("priority") == "high" else ""
        return {"response": prefix + reply, "trace": [f"respond: {team}"]}
    return handler

# Routing functions: read the state, return the NAME of the next node
def after_validate(state):
    return "intake" if state["validated"] else "reject"

def pick_team(state):
    return "legal" if state["compliance_flag"] else state["category"]

g = StateGraph(TriageState)
g.add_node("validate", v2_validate)
g.add_node("reject", v2_reject)
g.add_node("intake", v2_intake)
g.add_node("compliance", v2_compliance)
g.add_node("priority", v2_priority)
g.add_node("decide", lambda state: {})            # a join point for the two branches
g.add_node("billing", make_handler("billing", "Routed to Billing."))
g.add_node("technical", make_handler("technical", "Routed to Technical."))
g.add_node("general", make_handler("general", "Routed to General."))
g.add_node("legal", make_handler("legal", "Escalated to Legal/Compliance review.", use_prefix=False))

g.add_edge(START, "validate")                                  # cheap check first
g.add_conditional_edges("validate", after_validate, ["intake", "reject"])
g.add_edge("intake", "compliance")                             # fan out...
g.add_edge("intake", "priority")
g.add_edge(["compliance", "priority"], "decide")               # ...and join
g.add_conditional_edges("decide", pick_team, ["billing", "technical", "general", "legal"])
for node in ["reject", "billing", "technical", "general", "legal"]:
    g.add_edge(node, END)

app_v2 = g.compile()
```

```
for text in ["hi", "Our production server is down, this is urgent"]:
    out = app_v2.invoke({"ticket_id": "T-020", "text": text, "trace": []})
    print(out["response"], "|", out["trace"])
```

OUTPUT example: the model's wording and numbers will differ

```
Ticket rejected: not enough information. | ['validate: rejected', 'reject: too short']
[HIGH PRIORITY] Routed to Technical. | ['validate: passed', 'intake: technical (0.94)', 'compliance: False',
'priority: high', 'respond: technical']
```

Where this goes next: **checkpointing**. Give `compile()` a checkpointer and LangGraph saves the state after every node, so a run can pause for human approval and resume later, even after a crash. That's Session 2.

CHECKPOINT

1. What are the three moves to turn functions into a LangGraph app?
2. Why do LangGraph nodes take and return dicts instead of a Pydantic object here?
3. How would you find out which nodes ran for a particular ticket?

PART THREE

Prove It

Tests for code whose output isn't the same twice, and then your turn: a complete new path through the agent, built end to end and explainable line by line.

13 · Testing non-determinism 14 · Lab: add the urgent path

CHAPTER 13 SLIDE 32 1:20–1:30

Testing Non-Determinism

You can't assert the exact output of an LLM: the same input can get a slightly different answer twice. So test the constraints on the LLM parts, and test the plain code exactly, because it has no excuse.

LLM PARTS **Test the shape**

✓ category is one of the three
✓ confidence is between 0 and 1
✓ a reason is always attached

PLAIN CODE **Test exactly**

✓ "invoice" → billing fallback
✓ routing always picks the right team
✓ too-short tickets get rejected

Running pytest inside Colab

`pytest` normally runs test *files* in a separate process, which can't see the functions you defined in notebook cells. **ipytest** runs pytest on the current notebook instead. Run this once:

NOTEBOOK CELL · ENABLE PYTEST IN THE NOTEBOOK

main/m19_ipytest.py

```
import ipytest
ipytest.autoconfig()   # lets pytest find tests defined in notebook cells
```

Then any cell starting with `%%ipytest` becomes a test run. (`-vv` lists each test by name; ipytest is quiet by default.)

NOTEBOOK CELL · FOUR TESTS, TWO PHILOSOPHIES

main/m20_tests.py

```
%%ipytest -vv

def test_classification_returns_valid_category():
    # don't assert WHICH category on ambiguous text, the model's judgment can vary
    # assert the SHAPE is always valid instead
    result = classify_with_llm("My server keeps crashing and I was also charged twice")
    assert result.category in ("billing", "technical", "general")
    assert 0.0 <= result.confidence <= 1.0
    assert len(result.reasoning) > 0

def test_fallback_triggers_correctly():
    # this one IS deterministic, no LLM involved, so assert exactly
    result = fallback_keyword_classify("My invoice is wrong")
    assert result.category == "billing"
    assert result.reasoning.startswith("fallback")

def test_routing_is_deterministic():
    # routing has no LLM in it, test it exactly, every time, no exceptions
    state = TicketState(ticket_id="T-1", text="x", category="billing", validated=True)
    result = route(state)
    assert "Billing" in result.response

def test_short_ticket_is_rejected():
    result = validate(TicketState(ticket_id="T-2", text="hi"))
    assert result.validated is False
```

OUTPUT

your timing will differ

```
test_classification_returns_valid_category PASSED     [ 25%]
test_fallback_triggers_correctly PASSED               [ 50%]
test_routing_is_deterministic PASSED                  [ 75%]
test_short_ticket_is_rejected PASSED                  [100%]
================================== 4 passed, 2 warnings in 0.62s ===================================
```

You may also see a short "warnings summary" about plugins that were already imported. It's harmless.

What each test is really checking

TEST	KIND	WHY IT'S WRITTEN THIS WAY
`test_classification_returns_valid_category`	SHAPE	The ticket is deliberately ambiguous (a crash *and* a double charge). Either answer is defensible, so asserting one would make a **flaky** test. Asserting the shape catches the failures that matter.
`test_fallback_triggers_correctly`	EXACT	No LLM involved. Keyword logic either works or it doesn't.
`test_routing_is_deterministic`	EXACT	Routing is the rails: it must send billing to Billing every time, no exceptions.
`test_short_ticket_is_rejected`	EXACT	The gate's rule is the contract that protects everything downstream.

Four tests, two testing philosophies, one file. That distinction, knowing what you can and can't assert exactly, is what separates "I wrote some tests" from being able to defend your test suite in a code review.

THE FIRST TEST CALLS THE REAL API

It's slow (around a second), it uses your rate limit, and it fails if the Wi-Fi drops, even when your code is fine. Keep a few of these as **integration tests**, but don't let them be the only tests of your logic.

GO DEEPER · MOCK THE MODEL, THEN MEASURE IT

1. Unit tests with the LLM swapped out. `monkeypatch` temporarily replaces a function for one test, then puts it back. Swap the classifier for a stub and you can test `intake`'s logic exactly: fast, free and deterministic. `monkeypatch` is a built-in pytest *fixture*: name it as a test parameter and pytest passes it in. `setitem(globals(), …)` swaps the notebook-level name, because that's where `intake` looks it up.

PRODUCTION UPGRADE · MOCKING THE LLM deeper/d13_mock_tests.py

```
%%ipytest

def fake_classifier(category):
    return lambda text: TicketClassification(category=category, confidence=0.9, reasoning="stubbed
classifier")

def test_intake_copies_the_classification(monkeypatch):
    # replace the LLM call for the length of this test: fast, free, deterministic
    monkeypatch.setitem(globals(), "classify_with_retry", fake_classifier("technical"))
    state = intake(TicketState(ticket_id="T-1", text="it crashed"))
    assert state.category == "technical"
    assert state.confidence == 0.9

def test_compliance_beats_priority():
    state = TicketState(ticket_id="T-2", text="x", category="billing", validated=True,
                        compliance_flag=True, priority="high")
    assert route_v2(state).response == "Escalated to Legal/Compliance review."
```

OUTPUT

```
..                                                                       [100%]
2 passed, 2 warnings in 0.00s
```

2. Evals: is the model actually right? Tests check that code behaves; **evals** measure how often the model's judgement is correct. Keep a small, hand-labelled **golden set** and score against it whenever you change the prompt, the schema or the model:

PRODUCTION UPGRADE · A 10-TICKET GOLDEN SET deeper/d14_golden_set.py

```
GOLDEN = [
    ("My invoice #4521 charged me twice.", "billing"),
    ("I was charged twice this month", "billing"),
    ("Please refund my last payment", "billing"),
    ("My server keeps crashing after the last update", "technical"),
    ("The login page shows error 500", "technical"),
    ("The app is down and I need it fixed immediately", "technical"),
    ("How do I change my profile picture?", "general"),
    ("What are your support hours?", "general"),
    ("Do you have a student discount?", "general"),
    ("Can you send me a copy of last month's receipt?", "billing"),
]

def evaluate(classify, golden=GOLDEN):
    correct = 0
    for text, expected in golden:
```

```
        got = classify(text).category
        correct += got == expected
        if got != expected:
            print(f"MISS  {text!r}: got {got}, expected {expected}")
    print(f"accuracy: {correct}/{len(golden)} = {correct / len(golden):.0%}")

evaluate(classify_with_llm)
```

OUTPUT example: the model's wording and numbers will differ

```
MISS  "Can you send me a copy of last month's receipt?": got general, expected billing
accuracy: 9/10 = 90%
```

The miss is the interesting part: is a receipt request "billing" or "general"? Disagreements like this usually mean your **categories** need better definitions (Chapter 7's `Field(description=...)`), not that the model is broken. Evaluation gets a whole session later in the series.

CHECKPOINT

Shape test or exact test? Say why for each.

1. `fallback_keyword_classify("charged twice")` returns billing.
2. `classify_with_llm` on "the app crashes when I pay" returns a valid category.
3. `route_v2` escalates any ticket with the compliance flag set.

CHAPTER 14 SLIDE 33 1:30–1:55 · YOUR TURN · 25 MIN

Lab: Add the Urgent Path

Not three TODOs in a starter file: a complete new path through the agent, built end to end with every technique from today.

THE BRIEF

Start from the finished LangGraph app (Chapter 12): structured output, retries and a trace, all working. Add a fourth path: **urgent**. It needs its own schema value, its own node, its own routing branch, its own retry-safe classification, its own trace entries, and at least one test. When you're done, you should be able to explain every line of it in an interview.

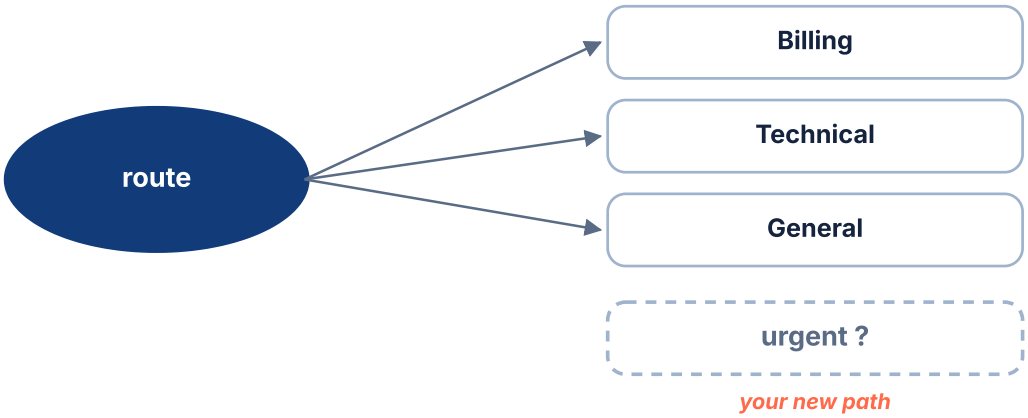


Figure 14.1 *Your target. What makes a ticket "urgent" is your call; one sensible definition is "a production outage, security incident or data loss happening right now".*

The checklist

- [ ] **Schema:** add "urgent" as a category the model can choose.
- [ ] **Node:** write the urgent handler, in the same shape as the other nodes.
- [ ] **Route:** urgent tickets actually take your new path.
- [ ] **Retry-safe:** it goes through retry and fallback; a flaky call on this path still falls back correctly.
- [ ] **Trace:** it logs like every other node.
- [ ] **Test:** write one test, and know why you picked that kind (shape or exact).

Stuck? Hints, one level at a time

Try for five minutes before reading a hint, and read only one level at a time. The struggle is where the learning happens.

Schema

HINT 1

The allowed categories live in exactly one place. Which class decides what the model is allowed to answer?

HINT 2

Add `"urgent"` to the `Literal` in `TicketClassification` and re-run that cell. `GraphState.category` is a plain `str`, so it doesn't need to change. Consider a `Field(description=...)` so the model knows what urgent *means* (Chapter 7, Go deeper).

Node

HINT 1

Copy the shape of `lg_route`: take the state dict, return `{**state, ...}` with a `response`.

HINT 2

Name it `lg_handle_urgent`. Register it with `graph.add_node(...)` next to the others.

Route

HINT 1

Every edge you've written so far is fixed: `add_edge(a, b)` always goes from a to b. For urgent tickets the graph has to *choose* between two possible next steps.

HINT 2

Look up `add_conditional_edges(source, path_function, mapping)`. The path function receives the state and returns a string; the mapping (a dict) turns that string into a node name. Chapter 12's Go deeper box uses the list form of the same call.

HINT 3

Decide *where* the choice happens: after which node do you know the category, the validation result and the compliance flag? And what should happen to an urgent ticket that also mentions a lawyer?

Retry-safe and trace

HINT 1

Your node doesn't call the LLM; `lg_intake` already does, through `classify_with_retry`. But what does the **fallback** return for "the site is down" when the LLM is unavailable?

HINT 2

Teach `fallback_keyword_classify` some urgent keywords, checked first. For the trace, append a line like `"handle_urgent: …"` to `state["trace"]` in your node's return value.

Test

HINT 1

Which part of your change has no LLM in it? That part deserves an **exact** test.

HINT 2

To test the wiring without depending on the model's judgement, stub the classifier with `monkeypatch` (Chapter 13, Go deeper), invoke the app, and assert on the response and the trace.

WHEN IT DOESN'T BEHAVE

Debug like you would in production. Print `result["trace"]` and ask whether every node you expected left its line. Then run `app.stream(ticket, stream_mode="updates")` to see exactly which nodes ran, in order. If you change a node function, rebuild and recompile the graph before testing again.

Definition of done

- An urgent ticket (try "Our production server is down, this is urgent") gets your urgent response, and your node's line appears in its trace.
- Billing, technical and general tickets still route exactly as before. Re-run your tests from Chapter 13.
- With a broken key, an outage ticket still ends up on the urgent path via the fallback.
- You can explain, without notes, why each line exists.

STRETCH GOALS · UPPER-YEAR

- Replace the `replies` dictionary in `lg_route` with one node per team and conditional edges (Figure 12.2).
- Only accept "urgent" when the model's confidence is at least 0.8; below that, send the ticket to a "needs human review" node. You've just built the hook Session 2 (human-in-the-loop) plugs into.
- Fix the substring bug from Chapter 10 with word-boundary regexes, and add a test proving "download" no longer counts as "down".
- Add three urgent tickets to the golden set (Chapter 13) and report the model's accuracy on them.

PART FOUR

Wrap-up & Reference

How to talk about what you built, the questions you'll be asked about it, and everything you'll want to look up later: a glossary, an error cheat-sheet and further reading.

15 · The Sentence 16 · Question bank A · Glossary B · Common errors cheat-sheet C · Further reading

CHAPTER 15 SLIDE 34 1:55–2:00

The Sentence

Not "I built a chatbot." This:

> **I designed a stateful agent with structured output, retry and fallback logic, and full tracing, then reimplemented it in LangGraph and tested the parts that needed testing.**

Write it down. That sentence is the real deliverable of the session. Every phrase in it is something you can now demonstrate and defend, which is what makes it worth saying.

THE PHRASE	WHAT YOU CAN SHOW	CHAPTER
stateful agent	A typed state object that every node reads and writes; the model makes one judgement and code does the rest	5, 8
structured output	A Pydantic schema with a closed set of categories, validated and re-asked on every call	7
retry and fallback logic	Bounded exponential backoff, plus a deterministic fallback that's clearly marked in the trace	9

THE PHRASE	WHAT YOU CAN SHOW	CHAPTER
full tracing	A trace entry from every step, timed JSON logs on the hand-built agent, and LangGraph's `stream()` view (the Go deeper version adds inputs and errors)	11, 12
reimplemented it in LangGraph	The same nodes wired with `add_node`, `add_edge` and `compile()`, and what the framework adds	12
tested the parts that needed testing	Shape tests for LLM output, exact tests for deterministic code, and why you chose each	13

On your resume

EXAMPLE BULLET

Built an LLM-powered support-ticket triage agent in Python: Pydantic-validated structured output via Instructor, a state-machine controller with retry, exponential backoff and keyword fallback, structured per-node tracing, and a LangGraph port; covered with pytest (shape tests for model output, exact tests for routing).

In a README

Push your notebook (without your key) to GitHub with a README that has five short sections: **What it does** (one paragraph and the diagram), **Architecture** (which node calls the LLM and why only that one), **Failure handling** (what happens when the API is down), **How to run it** (the install line and where the key goes), and **Tests** (what you test exactly vs. by shape). A reviewer should understand your design decisions in two minutes. For an upper-year portfolio piece, start from `triage_agent.py` (the production version shipped with this guide) and document what you changed.

The 60-second interview version

SAY IT OUT LOUD UNTIL IT'S NATURAL

"I built a support-ticket triage agent. The LLM does one thing: it classifies the ticket into a fixed schema, so my router can trust its output. Everything else is deterministic code: validation, compliance and priority checks, and routing through a lookup table. LLM calls fail, so classification retries with exponential backoff and falls back to keyword matching, and the fallback is marked in the trace. Every step leaves a trace entry, and the model call logs its decision, reasoning and latency, so I can debug any wrong call. I built it by hand first, then ported it to LangGraph, which is how I know what the framework actually does for me. For tests, I check the shape of model output and test the deterministic parts exactly."

CHAPTER 16 AT WORK SLIDES REVIEW

Question Bank

Every At Work question from the deck and this guide, plus the follow-ups you're likely to get. Cover the

answer and say yours first.

MANAGER "Can we just let the bot answer the refund questions?"

Not on its own. It predicts plausible text; it doesn't know our policy. Ground it by putting the real policy in context, constrain what it can say, and send anything about money that isn't covered by the policy to a human.

INTERVIEW "Why do LLMs hallucinate?"

They generate the most likely next token, not verified facts. When the answer isn't in the context, plausible wins. Mitigate by grounding, constraining output, and verifying before acting.

MANAGER "Let's build a fully autonomous support agent."

First map the steps. If they're known in advance, use a state machine with the LLM in the one node that needs judgement. Keep autonomy for the parts where you genuinely can't write the path.

INTERVIEW "When would you not build an agent?"

When the steps are known up front. Workflows are cheaper, faster, more predictable and testable; agents trade all of that for flexibility you may not need.

MANAGER "The dashboard crashed: the bot replied 'Sure! Here is your JSON...'"

We were parsing free text. Define the output as a schema, validate every response against it, and re-ask automatically on a mismatch.

INTERVIEW "How do you get reliable JSON from an LLM?"

A Pydantic schema passed as `response_model` through a library like Instructor, which validates and retries. Shape is guaranteed; judgement isn't, so you still test and monitor correctness.

MANAGER "Our LLM bill tripled. Why?"

Find every step that calls the model. Turn rule-shaped steps into plain code, move cheap gates before the model call, and keep the LLM for judgement only.

INTERVIEW "What happens when your LLM provider goes down?"

Bounded retries with exponential backoff (and jitter) for temporary errors, fail fast on permanent ones like a bad key, then a deterministic fallback that's marked in the logs. The request degrades; it never crashes.

INTERVIEW "What's in a context window, and what if you exceed it?"

Everything the model sees in one call, measured in tokens. Over the limit, the request fails or older content has to be dropped or summarised. Anything not in the window doesn't exist to the model.

INTERVIEW "Which temperature would you use for classification?"

Low, close to zero, for consistency. Even then, outputs can vary slightly, so don't depend on exact wording.

INTERVIEW "Is the model's confidence score reliable?"

It's self-reported, not a calibrated probability. Treat it as a weak signal: useful for thresholds and human review, but validate it against a labelled set before relying on it.

INTERVIEW "Instructor's `max_retries` vs your retry loop: what's the difference?"

`max_retries` re-asks the model when its output fails validation. The retry loop handles the call itself failing (rate limits, outages). The SDK has a third layer of its own for network errors.

INTERVIEW "What does a reducer do in LangGraph?"

It defines how multiple updates to the same state key are combined, for example concatenating lists from parallel branches. Without one, two branches writing the same key in one step is an error.

INTERVIEW "Why use LangGraph if plain Python works?"

For what comes next: conditional and parallel edges, safe state merging, checkpointing and resume, human-in-the-loop interrupts, streaming and visualisation. Building it by hand first is how you know which of those you need.

INTERVIEW "How do you test non-deterministic code?"

Assert invariants on model output (valid category, bounded confidence, reasoning present), test deterministic code exactly, mock the model for unit tests, and run evals on a labelled golden set to measure accuracy.

INTERVIEW **"How would you debug an agent that made the wrong call?"**

Pull the trace for that run, find the node that made the decision and the input it saw, reproduce it in isolation, fix the prompt, schema, rule or ordering, and add the case as a regression test.

INTERVIEW **"Workflow vs agent: what's the difference?"**

In a workflow, code defines the path and LLMs fill in steps. In an agent, the LLM decides its own next steps and tool use. Start with workflows; add autonomy only when the path can't be written down.

INTERVIEW **"How would you cut latency?"**

Measure with traces first: the LLM call dominates. Use fewer calls, a smaller or faster model where accuracy allows, shorter prompts and outputs, parallel independent steps, and gates before the model.

APPENDIX A REFERENCE

Glossary

Agent
An LLM whose output is a decision that code acts on, usually in a loop. Today: an LLM in one node of a state machine.

Agent loop
Observe → think → act, repeated until done. Only "think" is an LLM call.

API key
A secret string that identifies your account to a provider. Lives in Colab Secrets or an environment variable, never in code.

Backoff (exponential)
Waiting longer after each failed attempt (1s, 2s, 4s…) so you don't hammer a struggling service.

Circuit breaker
Stops calling a failing service for a while after repeated errors, then tests it again.

compile()
Turns a LangGraph `StateGraph` blueprint into a runnable app, checking the wiring as it goes.

Conditional edge
A LangGraph edge whose destination is chosen at run time by a function that reads the state.

Context window
Everything a model can see in one call, measured in tokens. Outside it, nothing exists to the model.

Controller
The code that decides which step runs next. In LangGraph, the compiled graph plays this role.

Decorator
A function that wraps another function to add behaviour (timing, logging) without changing it.

Dispatch table
A dictionary from a key (like a category) to the function that handles it. It replaces long `if/elif` chains.

Edge
A connection between nodes in a graph: "after this, go there".

Eval / golden set
Measuring how often the model is *correct*, against a small hand-labelled dataset.

Fallback
A simpler path used when the main one fails. Here, keyword matching when the LLM is unavailable.

Graceful degradation
Giving a worse answer instead of no answer (or a crash) when something fails.

Grounding
Putting real facts (policies, documents, data) into the context so the model doesn't rely on memory.

Hallucination
Fluent, confident output that isn't true, a natural result of predicting plausible text.

Instructor
A library that makes LLMs return validated Pydantic objects via `response_model`, re-asking on failure.

Jitter
A small random addition to retry waits so many clients don't retry at the same instant.

LangGraph
A framework for building agents as graphs of state, nodes and edges, with persistence, streaming and human-in-the-loop support.

Literal
A type hint meaning "exactly one of these values", which turns a free-text field into a closed set.

Mock / monkeypatch
Temporarily replacing a function (like the LLM call) in a test with a predictable stand-in.

Node
One step in a graph: a function that takes the state and returns an update.

Observability
Being able to tell what a running system did and why, from its logs and traces.

Pipeline
Fixed steps in a fixed order, every time. No judgement involved.

Pydantic
A Python library that validates data against classes declared with type hints.

Rate limit (429)
A provider's cap on requests or tokens per minute or day. Exceeding it returns HTTP 429.

Reducer
The rule for merging several updates to the same piece of state, for example concatenating trace lists from parallel branches.

Router / rails
The LLM classifies (the router); deterministic code moves the work (the rails).

Schema
A precise description of a data shape: fields, types and allowed values.

State
The single record of everything known about the current job, read and written by every node.

State machine
A system that is always in a known state and moves between states by explicit rules.

Structured output
Model output forced into a schema, so code can rely on its shape.

Temperature
How random the model's choice of next token is. Low means consistent, high means varied.

Token
The unit models read and write, roughly ¾ of an English word. Limits and costs are counted in tokens.

Tool calling
A model "calling" a function by returning its name and JSON arguments. Instructor uses it to fill schemas.

Trace
A per-run record of what each step received, decided and how long it took.

TypedDict
A dictionary type with declared keys and value types, used for LangGraph state.

Workflow automation
Event-triggered, rule-based steps (think Zapier). No LLM.

Workflow (Anthropic's sense)
Any system where code defines the path and LLMs fill in steps. Today's agent is a routing workflow.

APPENDIX B REFERENCE

Common Errors Cheat–Sheet

Read the **last line** of a traceback first: it names the error. Then find it here. Setup errors (secrets, install, keys) are in Chapter 0.

YOU SEE	LIKELY CAUSE AND FIX
`NameError: name '…' is not defined`	A cell above wasn't run in this session, or ran out of order. **Runtime → Run before** on your current cell.
`InstructorRetryException`	Instructor gave up. The real cause is in `e.__cause__` : often a 401 or 429, or a model that keeps breaking the schema (try a bigger model).
`ValidationError` creating your own object	You passed a value the schema doesn't allow. The message names the field and the expected values.
A cell pauses for several seconds, printing `attempt … failed`	The retry loop is backing off. Read the error inside the brackets: if it's a 401, retrying won't help.

YOU SEE	LIKELY CAUSE AND FIX
`KeyError: '…'` inside a routing function	A dispatch table has no entry for that value. Print the state's category and check every possible value has a destination.
`AttributeError: 'dict' object has no attribute 'text'`	LangGraph state is a dict: use `state["text"]`, not `state.text`.
`TypeError: 'TicketState' object is not subscriptable`	The reverse: a Pydantic object. Use `state.text`.
`AttributeError: 'StateGraph' object has no attribute 'invoke'`	You forgot `app = graph.compile()`. Invoke the compiled app.
`ValueError: Found edge ending at unknown node`	A typo in a node name inside `add_edge`. Edges use node *names*, as given to `add_node`.
`InvalidUpdateError: … Can receive only one value per step`	Two nodes ran in the same step and wrote the same key. Either they're parallel branches (add a reducer: `Annotated[list, operator.add]`, Chapter 10), or one node has *both* a fixed and a conditional edge out. There's no `remove_edge`, so rebuild the graph from a fresh `StateGraph`.
Warning: "Adding a node to a graph that has already been compiled"	You changed the blueprint after compiling. Re-run the whole graph-building cell so `compile()` runs again.
A node's change "doesn't stick" in LangGraph	The node didn't `return` its update (LangGraph treats `None` as "no change"), or the graph is stale. Return the dict and recompile.
`UsageError: Cell magic `%%ipytest` not found`	Run the `ipytest.autoconfig()` cell first, and make sure `%%ipytest` is the very first line of the cell.
Your LLM output doesn't match the guide	Expected: wording and confidence vary from run to run. Only worry if the *structure* differs (missing fields, errors).
`Error code: 429` in the middle of the lab	You're over the free tier's per-minute limit. Wait 60 seconds; stop re-running cells in quick succession.

APPENDIX C REFERENCE

Further Reading

- **Anthropic, "Building effective agents"** (Dec 2024): the clearest short read on workflows vs agents and when to use which. anthropic.com/engineering/building-effective-agents
- **LangGraph docs**: state, nodes, edges, reducers, and the persistence and human-in-the-loop features coming in Session 2. docs.langchain.com/oss/python/langgraph
- **Instructor docs**: `response_model`, validators, retries and modes, with examples for every provider. python.useinstructor.com
- **Pydantic docs**: models, `Field`, validators, `ConfigDict`. docs.pydantic.dev
- **Groq docs**: current models, rate limits, tool use. console.groq.com/docs · **ipytest**: pytest inside notebooks. github.com/chmp/ipytest
- ***Moffatt v. Air Canada*, 2024 BCCRT 149**: the chatbot case from Chapter 3 (short and readable). **Russell & Norvig, *AI: A Modern Approach***, ch. 2: the classic agent types.

APPENDIX D | REFERENCE | CHECKED 9 OCT 2026

What Employers Actually Ask For

It's easy to say "companies want this". It's more useful to check. We opened 17 live AI-engineering job postings and counted which of today's skills each one asks for.

On **9 October 2026** we collected 17 postings for roles like AI Engineer, Applied AI Engineer and Agentic AI Engineer from across North America. **9 of them are open to candidates in Canada** (Toronto, Montreal, Vancouver and remote-Canada roles). Every posting was opened on the employer's own careers page, and a skill only counts when the posting says it in its own words.



Figure D.1 *How many of 17 live postings ask for each skill, and the chapter that teaches it. Grey: not part of Session 1.*

What the numbers say

- **Agents and orchestration are near-universal (16 of 17).** The one exception is a classic machine-learning role.
- **Evals (15) and observability (13) beat frameworks (9).** Employers ask "can you measure it and debug it?" more often than they ask for any library. That's why Chapters 11 and 13 matter so much.
- **Reliability appears in 11, often in plain words.** Machinify writes "retries, fallbacks, cost controls"; Future writes "retry logic".
- **LangGraph is the most-named framework** (8 postings), then LangChain (7). CrewAI, LlamaIndex, Vercel AI SDK, CopilotKit, Temporal and Inngest appear once each.
- **Structured output appears in 7,** usually phrased as "tool calling", "function calling" or "structured outputs". Only Kobie names Pydantic, but every one of those tools works the way Chapter 7 does.
- **RAG appears in 11.** It isn't in Session 1, which tells you where later sessions are heading.

The skill on the page

"Experience building systems that turn messy, unstructured source material into validated structured output"

Netomi, Staff Agentic AI Engineer, Toronto

The chapter that teaches it

Chapter 7: a Pydantic schema passed as `response_model`, validated on every call and re-asked when it doesn't fit.

The 17 postings

S = structured output · A = agents/orchestration · R = reliability · O = observability · E = testing/evals · F = names an agent framework. Roles open to candidates in Canada are listed first. Links were live on 9 October 2026.

COMPANY	ROLE	LOCATION	S	A	R	O	E	F
Clio	Machine Learning Engineer	Canada (Burnaby, Calgary, Toronto)		✓				
Guidepoint	AI Engineer	Toronto (hybrid)		✓	✓	✓	✓	✓
Honeycomb	Senior SWE II, Agentic Intelligence	Remote, Canada		✓		✓	✓	
Later	Senior AI Automation Engineer	Toronto, Vancouver + US		✓	✓	✓		
Maple	Staff AI Engineer	Toronto / remote Canada		✓	✓	✓	✓	✓
Mistral AI	Applied AI, Forward Deployed MLE	Montreal (hybrid)		✓			✓	✓
NerdWallet	Software Engineer, AI & Automation	Remote, Canada		✓	✓	✓	✓	
Netomi	Staff Agentic AI Engineer	Toronto / remote Canada	✓	✓	✓	✓	✓	✓
Samsara	AI Architect, GTM Systems	Remote, Canada	✓	✓		✓	✓	✓
Anthropic	Applied AI Engineer, Enterprise Tech	SF / NYC / Seattle		✓	✓		✓	
Bolt.new (StackBlitz)	Staff Applied AI Engineer	Remote	✓	✓	✓		✓	✓
Future	Applied AI Engineer	Remote (US)	✓	✓	✓	✓	✓	✓
Glean	ML Engineer, LLM Evals & Observability	SF Bay Area		✓		✓	✓	
Gusto	Staff Applied AI/ML Engineer	US + Toronto (remote)				✓	✓	
Kobie	AI Engineer	Remote (US)	✓	✓	✓	✓	✓	✓
Machinify	Staff AI Engineer	US	✓	✓	✓	✓	✓	✓
WITHIN	Member of Technical Staff, AI	New York	✓	✓	✓	✓	✓	

One line from each posting

Exact quotes, useful for seeing how employers phrase these skills, and for borrowing their vocabulary honestly in your own résumé.

- **Clio:** “Experience with agentic frameworks and workflows”
- **Guidepoint:** “Build agents that handle intent interpretation, task decomposition, tool use, and human escalation”
- **Honeycomb:** “measurable against real evals, maintainable, and honest about their limits”

- **Later:** “lets automations run unattended and recover gracefully when they don't”
- **Maple:** “A strong track record designing evaluation strategies for non-deterministic systems”
- **Mistral AI:** “experience in Fine Tuning LLMs, tackling advanced RAG or agentic use cases”
- **NerdWallet:** “You do not need to come from a traditional security engineering background.”
- **Netomi:** “Experience building systems that turn messy, unstructured source material into validated structured output”
- **Samsara:** “Experience building evals and observability tooling for LLM-powered systems”
- **Anthropic:** “documenting the latest LLM prompting, eval, agentic, and architecture techniques”
- **Bolt.new (StackBlitz):** “Design safe, reliable interfaces for tool calling”
- **Future:** “Improve reliability, latency, and cost through prompt caching strategies, token budgets, retry logic, and observability.”
- **Glean:** “providing the quality signal that gates launches and prevents regressions”
- **Gusto:** “familiarity with best engineering practices (CI/CD, testing, code review)”
- **Kobie:** “structured outputs (Pydantic/JSON schema), retries, streaming, and memory”
- **Machinify:** “can read a token bill and a trace and know where to cut”
- **WITHIN:** “agent workflows that use tools, call external systems, maintain state, and handle multi-step tasks”

JOB LINKS DIE

Postings close within weeks, so some links above will stop working. That's why the quotes are printed here. To repeat the exercise yourself, search Lever, Greenhouse and Ashby boards for "AI Engineer" or "Applied AI Engineer", paste each posting's requirements into a spreadsheet, and tally the skills.

How to use this when you apply

1. **Match their words, honestly.** If a posting says "evaluation harnesses" and you wrote shape tests plus a golden set (Chapter 13), say "built an evaluation harness with shape tests and a labelled golden set". Never claim what you didn't build.
2. **Lead with the rarer skills.** Everyone claims "agents". Fewer applicants can talk concretely about tracing, fallbacks and evals, which is what 11–15 of these postings ask for.
3. **Bring evidence.** A public repo with a README, a trace and passing tests (Chapter 15) is worth more than any adjective.
4. **Have one bug story ready.** "Tell me about a bug you fixed" is almost guaranteed. The lab gives you one.

YOUR BUG STORY, IN STAR FORM

Situation: I was adding an urgent path to a LangGraph triage agent. **Task:** make urgent tickets reachable without breaking the other routes. **Action:** the run failed with `KeyError: 'urgent'`; the trace stopped right after the routing step, so I checked the routing table and found the new category had a node but no entry in the edge mapping. I built the mapping from the same dictionary as the nodes so they can't drift apart. **Result:** urgent tickets route correctly, even through the fallback during an outage, and a test now guards it.

APPENDIX E REFERENCE

Explain It Simply

Every idea from today, in one everyday picture. Use these when a classmate looks lost, or when an interviewer asks you to explain something "to a non-technical stakeholder".

THE IDEA	SAY IT LIKE THIS	CH.
The kernel	A chef with a notepad: it remembers every instruction in the order it arrived, not the order they're pinned on the wall.	P
An API call	A drive-through. You speak into the box, a kitchen you can't see does the work, the order comes out the window. The key is your loyalty card.	P
Next-token prediction	Your phone's autocomplete, trained on a huge slice of the internet and run very fast. It knows which word usually comes next, not which word is true.	2
Temperature	Low: order your usual. High: close your eyes and point at the menu. For routing tickets you want your usual.	2
No memory between calls	Every call is a new phone call to someone who's never heard of you. Chat apps "remember" by re-reading the whole conversation to them every time.	2
Workflow vs agent	A train versus a self-driving car. The car can go anywhere, which is exactly why nobody fully trusts it. The train only chooses at switches. Most companies ship trains.	4, 5
LLM as router, code as rails	A triage nurse decides what kind of case you are. Hospital rules decide which ward, and the nurse never wheels you there.	5
Structured output	A web form with dropdowns instead of a blank essay box. Same customer, far less chaos.	7
State and the trace	A patient's chart on a clipboard. Each station does one job, writes its result and initials it. The chart is the state; the initials are the trace.	8
Backoff and fallback	A friend who doesn't pick up: you wait a bit, then longer, instead of redialling every second. If they still don't answer, you text. The text is the fallback.	9
Branches and the reducer	Security and customs each stamp the same passport. The gate agent reads every stamp and makes one decision.	10
A decorator	A phone case: it adds grip, a wallet and a stand without opening the phone.	11
Observability	Parcel tracking for your agent: every stop, when it got there, and what happened to it.	11
LangGraph	A subway map: stations are nodes, tracks are edges, and one transfer station has a sign that says which line to take.	12
Exact vs shape tests	A vending machine versus marking an essay. B4 gives chips every time, so test it exactly. An essay can't be marked on exact words, only on whether it answers the question.	13

APPENDIX F | LAB SOLUTION | READ AFTER YOU TRY

Lab Solution: The Urgent Path

The complete worked solution to Chapter 14, step by step, with the reasoning behind every line. Try the lab first: you'll learn far more from reading this after you've been stuck than before.

This solution starts from the finished LangGraph app in Chapter 12 (`main/m17_langgraph.py`). Every cell below was run end to end before this guide shipped: normal routes, the urgent path, an urgent ticket that mentions a lawyer, and an urgent ticket during an API outage.



Figure F.1 *The finished graph. Solid arrows are `add_edge`; the dashed choice after `priority` is `add_conditional_edges`. Both leaves go to END.*

Step 1 · The schema: teach the model what "urgent" means

The allowed categories live in exactly one place, `TicketClassification`, so that's the only class that changes. `GraphState.category` is a plain `str`, so it needs nothing.

LAB SOLUTION · SCHEMA — lab/l01_schema.py

```
from typing import Literal
from pydantic import BaseModel, Field

class TicketClassification(BaseModel):
    category: Literal["billing", "technical", "general", "urgent"] = Field(
        description=(
            "urgent: a production outage, security incident or data loss happening right now. "
            "technical: a bug or error that is not an outage. "
            "billing: charges, invoices, refunds, receipts. "
            "general: anything else."
        )
    )
    confidence: float
    reasoning: str
```

- **Why the description?** "Our production server is down" is *also* a technical problem. Without a definition, the model's choice between technical and urgent is close to a coin flip. Instructor sends field descriptions to the model as part of the schema, so this is how you define the category it would otherwise confuse.
- **Re-run the cells that use the class.** `classify_with_llm` and `fallback_keyword_classify` build `TicketClassification` objects, so re-run their cells after this one, or they keep the old three-value version.

Step 2 · The fallback must know about urgent too

Picture an outage *during* an outage: the API is down, so every ticket goes through the keyword fallback. If the fallback doesn't know about urgent, a production-down ticket quietly lands in technical at exactly the moment it matters most.

```python
import re

URGENT_PATTERN = re.compile(r"\b(urgent|outage|asap|immediately|down)\b")

def fallback_keyword_classify(text: str) -> TicketClassification:
    t = text.lower()
    if URGENT_PATTERN.search(t):              # urgent FIRST: an outage often also says "crash"
        cat = "urgent"
    elif "invoice" in t or "charge" in t:
        cat = "billing"
    elif "error" in t or "bug" in t or "crash" in t:
        cat = "technical"
    else:
        cat = "general"
    return TicketClassification(category=cat, confidence=0.5,
                                reasoning="fallback: keyword match, LLM unavailable")
```

- **Urgent is checked first** because Python runs the first matching branch, and "the server is down and keeps crashing" would otherwise stop at technical.
- `\b` **means "word boundary".** It fixes the substring bug from Chapter 10: `"down" in t` is true for "download" and "countdown", but `\bdown\b` only matches the word "down". So "I can't download my invoice" stays billing.
- `classify_with_retry` doesn't change at all. It already falls back to this function, so the urgent path is retry-safe for free.

Step 3 · The urgent node

Same shape as every LangGraph node: take the state dict, return a copy with the keys you change. It also adds its own trace line, which is one of the checklist items.

LAB SOLUTION · NODE — lab/l03_node.py

```python
def lg_handle_urgent(state: GraphState) -> GraphState:
    return {**state,
            "response": "[URGENT] Paged on-call. Incident opened.",
            "trace": state["trace"] + ["handle_urgent: paged on-call, incident opened"]}
```

`state["trace"] + [...]` builds a *new* list instead of calling `.append()` on the old one. Chapter 12's pitfall explains why: never mutate the input, return the update.

Step 4 · The routing function and the rebuilt graph

Every edge so far was fixed. Urgent tickets need the graph to **choose**, so the fixed edge `priority → route` becomes a conditional edge. The choice happens **after** `priority`, because that's the first point where the category, the validation result and the compliance flag are all known.

```python
def pick_path(state: GraphState) -> str:
    # Legal beats urgent: a ticket that mentions a lawyer goes to route, which escalates it.
    if state["validated"] and not state["compliance_flag"] and state["category"] == "urgent":
        return "urgent"
    return "normal"

graph = StateGraph(GraphState)                # a FRESH graph: there is no remove_edge
graph.add_node("intake", lg_intake)
graph.add_node("validate", lg_validate)
graph.add_node("compliance", lg_compliance)
graph.add_node("priority", lg_priority)
graph.add_node("route", lg_route)
graph.add_node("handle_urgent", lg_handle_urgent)

graph.set_entry_point("intake")
graph.add_edge("intake", "validate")
graph.add_edge("validate", "compliance")
graph.add_edge("compliance", "priority")
graph.add_conditional_edges("priority", pick_path,
                            {"urgent": "handle_urgent", "normal": "route"})
graph.add_edge("route", END)
graph.add_edge("handle_urgent", END)

app = graph.compile()
result = app.invoke({"ticket_id": "T-007", "text": "Our production server is down, this is urgent", "trace":
[]})
print(result["response"])
print(result["trace"])
```

OUTPUT example: the model's wording and numbers will differ

```
[URGENT] Paged on-call. Incident opened.
['intake: urgent (confidence=0.95)', 'handle_urgent: paged on-call, incident opened']
```

- **`pick_path` returns a word, and the mapping turns the word into a node.** `"urgent"` goes to `handle_urgent`; everything else goes to the old `route` node, which still handles billing, technical, general, rejected and legal exactly as before.
- **Legal beats urgent.** "The server is down and my lawyer is involved" goes to `route`, which escalates it to Legal. That's a policy decision, made in one readable line.
- **Why not just add "urgent" to `lg_route`'s replies?** It would work, but the urgent path would be invisible: no separate node in the drawing, no separate step in `stream()`. Giving it its own node makes the path show up in the graph, which is the point of using LangGraph.

Step 5 · The tests

Four tests. Three are **exact**, because routing, the node and the fallback contain no LLM. The fourth tests the **wiring** end to end with the classifier stubbed out (Chapter 13, Go deeper), so it's fast, free and doesn't depend on the model's judgement.

LAB SOLUTION · TESTS

lab/l05_tests.py

```
%%ipytest -vv

def fake_classifier(category):
    return lambda text: TicketClassification(category=category, confidence=0.9, reasoning="stubbed")

def test_pick_path_sends_urgent_to_its_node():
    state = {"validated": True, "compliance_flag": False, "category": "urgent"}
    assert pick_path(state) == "urgent"

def test_legal_beats_urgent():
    state = {"validated": True, "compliance_flag": True, "category": "urgent"}
    assert pick_path(state) == "normal"

def test_fallback_knows_urgent_but_not_download():
    assert fallback_keyword_classify("Total outage, fix this ASAP").category == "urgent"
    assert fallback_keyword_classify("I can't download my invoice").category == "billing"

def test_urgent_wiring_end_to_end(monkeypatch):
    monkeypatch.setitem(globals(), "classify_with_retry", fake_classifier("urgent"))
    out = app.invoke({"ticket_id": "T-U", "text": "Our production server is down", "trace": []})
    assert out["response"].startswith("[URGENT]")
    assert any(line.startswith("handle_urgent") for line in out["trace"])
```

OUTPUT

your timing will differ

```
test_pick_path_sends_urgent_to_its_node PASSED          [ 25%]
test_legal_beats_urgent PASSED                          [ 50%]
test_fallback_knows_urgent_but_not_download PASSED      [ 75%]
test_urgent_wiring_end_to_end PASSED                    [100%]
4 passed
```

The stub works because `lg_intake` looks up `classify_with_retry` by name each time it runs, so swapping the notebook-level name for the length of one test changes what the compiled graph calls. If you'd rather keep one test on the real model, add a **shape** test: the answer for an outage ticket must be one of the four valid categories.

Step 6 · Check the definition of done

LAB SOLUTION · DEFINITION OF DONE

lab/l06_done.py

```
# 1. Every old route still works
for text in ["My invoice #4521 charged me twice.", "My server keeps crashing after the last update",
             "What are your office hours?", "hi", "Server is down and urgent, my lawyer is involved"]:
    print(app.invoke({"ticket_id": "T", "text": text, "trace": []})["response"])

# 2. With a broken key, an outage ticket still takes the urgent path, via the fallback
client = instructor.from_groq(Groq(api_key="gsk_not_a_real_key"))
out = app.invoke({"ticket_id": "T", "text": "Our production server is down, this is urgent", "trace": []})
print(out["response"], out["trace"])
client = instructor.from_groq(groq_client)    # put the real client back!
```

OUTPUT

example: retry messages trimmed

```
Routed to Billing.
Routed to Technical.
Routed to General.
Ticket rejected: not enough information.
Escalated to Legal/Compliance review.
all 3 attempts failed, falling back to keyword matching
[URGENT] Paged on-call. Incident opened. ['intake: urgent (confidence=0.50)', 'handle_urgent: paged on-call,
incident opened']
```

`confidence=0.50` in the trace is the fallback telling on itself: the model was unreachable, the keyword rules caught the outage, and the ticket still reached on-call.

The bugs people actually hit

WHAT YOU SEE	WHAT'S WRONG	THE FIX
`KeyError: 'urgent'` raised inside `lg_route`	You added "urgent" to the schema, but the graph still has the fixed edge `priority → route`, so `lg_route` looks up `replies["urgent"]`	Give urgent its own path: replace the fixed edge with `add_conditional_edges` (Step 4)
`KeyError: 'urgent'` even though you added a conditional edge	The routing function returns "urgent", but the mapping dict has no "urgent" key	Every word the routing function can return needs an entry in the mapping
The outage ticket ends up in **technical**	No `Field(description=...)`, so the model can't tell urgent from technical	Define each category in the schema (Step 1)
Urgent works normally, but goes to technical during an outage	The fallback doesn't know about urgent, or checks "crash" before urgent words	Teach the fallback urgent words, checked first (Step 2)
"I can't download my invoice" is flagged urgent	`"down" in t` matches inside "download"	Word boundaries: `\bdown\b` (Step 2)
Warning: adding a node to a graph that's already compiled	You added to the old `graph` object instead of building a new one	Start from a fresh `StateGraph(GraphState)` and re-run the whole cell

DEBUG IT LIKE PRODUCTION

Both `KeyError`s are found the same way: read the trace before you read your code. The last line is `intake: urgent (...)` and nothing after it, so the problem is in whatever runs *after* the graph knows the category: the routing step. `app.stream(ticket, stream_mode="updates")` shows the exact nodes that ran. Turn this into your interview bug story with the STAR template in Appendix D.

WHAT'S NEXT

Session 2: Human in the Loop

Today's agent decides and acts on its own. Next time, it learns to stop and wait for a person before it does anything risky, without losing its place.

Dalhousie Machine Learning Society

dmls@dal.ca · Instagram @dalmlsociety

Learn. Build. Grow. With DMLS.